

OS pour le déploiement de services conteneurisés



Thèse de Bachelor présentée par

ARROYO FRÉDÉRIC

pour l'obtention du titre de Bachelor of Science HES-SO en

**Informatique et systèmes de communication avec orientation en
Informatique logicielle**

Septembre 2026

Professeur HES responsable

GLÜCK FLORENT

Légende et source de l'illustration de couverture:

La mascotte de Linux à côté d'un conteneur cargo, Swapneel MEHTA pour [OpenSourceForU.com](https://opensourceforu.com)

Table des matières

Remerciements	VII
Énoncé du sujet	VIII
Résumé	IX
Glossaire	X
Introduction	1
1. Présentation du système	5
1.1. Vue d'ensemble	5
1.2. Ressources	6
1.3. Déclarativité et immutabilité	7
1.4. Administration du système et API	8
1.5. Installation du système et modes de fonctionnement	9
1.6. Conteneurs	10
1.7. Prérequis matériels	11
1.8. Exemple d'utilisation	11
2. Conception du système	14
2.1. Composants du système	14
2.2. Modèle de ressource	15
2.2.1. Structure d'une ressource	15
2.2.2. Relations entre les ressources	15
2.2.3. Catégories de ressources	15
2.2.4. Ressources mutualisées	18
2.3. Orchestrateur et réconciliation	22
2.3.1. Orchestrateur	22
2.3.2. Contrôleur et déclarativité	22
2.3.3. Orchestration de la réconciliation	23
2.3.4. Boucle de réconciliation	27
2.3.5. Suppression des ressources	27
2.3.6. Réaction aux événements	28
3. Implémentation	29
3.1. Composants	29
3.1.1. Choix des logiciels tiers	29
3.1.2. Choix du langage	29
3.1.3. Choix des bibliothèques	29
3.1.4. Communication entre les composants	30
3.1.5. Sécurité et isolation	31

3.2.	Orchestrateur	31
3.2.1.	File d'attente	32
3.2.2.	Réconciliation	34
3.2.3.	Validation d'une ressource	36
3.3.	Contrôleurs	36
3.3.1.	Contrôleur système	38
3.3.2.	Contrôleur réseau	38
3.3.3.	Contrôleur de conteneurs	41
3.4.	API et clients d'administration	41
3.5.	Système de fichier racine	42
3.6.	Démarrage du système	42
3.7.	Installation et chiffrement	45
3.8.	Système de build	45
3.9.	Configuration du noyau	46
3.10.	Génération de l'image du système	46
3.11.	Environnement de test	47
3.12.	Pipeline CI/CD	48
4.	Tests et validation	51
4.1.	Tests unitaires et tests d'intégration	51
4.2.	Validation	52
4.2.1.	Exécution dans un environnement éphémère	52
4.2.2.	Installation du système	52
4.2.3.	Application 3 tiers	52
4.3.	Analyse de performance	52
4.3.1.	Rapidité	53
4.3.2.	Légèreté	55
4.4.	Exécution sur un Raspberry Pi 1B	56
4.5.	Limitations	57
5.	Comparaison avec les solutions existantes	58
5.1.	Critères	58
5.1.1.	Automatisation	58
5.1.2.	Légèreté	58
5.1.3.	Rapidité	58
5.1.4.	Simplicité	58
5.2.	Méthodologie	59
5.3.	NixOS	59
5.4.	Talos Linux	60

5.5. Synthèse	60
6. Résultats et Discussion	62
6.1. Résultats fonctionnels	62
6.2. Résultats académiques	63
6.3. Difficultés rencontrées	65
6.4. Rétrospective	66
6.5. Perspectives	67
Conclusion	68
Annexe A – Utilisation de l’intelligence artificielle	71
A.1. Recherche d’options de configuration manquantes du noyau Linux	71
A.2. Amélioration du pipeline de build Nix pour Rust	71
A.3. Mise en place de tests d’intégration Rust isolés par namespace	72
Annexe B – Exemple de configuration complète de l’OS	73
Annexe C – Le langage Nix	74
C.1. Dérivations	74
C.2. nixpkgs	74
C.3. Le Nix Store	74
C.4. Nix comme système de build	74
Bibliographie	76

Table des illustrations

Figure 1 Réconciliation d’une ressource	14
Figure 2 Création de sous-ressources à partir d’une configuration utilisateur	17
Figure 3 Création de sous-ressources à partir d’une configuration DHCP	18
Figure 4 Problème des ressources partagées implicitement	19
Figure 5 Solution au problème du partage de ressources implicite	21
Figure 6 Réconciliation d’une ressource	23
Figure 7 Orchestration de la réconciliation dans un modèle centralisé	24
Figure 8 Orchestration de la réconciliation dans un modèle décentralisé	24
Figure 9 Démarrage du système	43
Figure 10 Panique du noyau lorsqu’un disque n’est pas disponible	44
Figure 11 Arbre d’exécution des processus du système	44
Figure 12 Arbre de build	47
Figure 13 Pipeline CI/CD	49
Figure 14 Chronologie des étapes de démarrage jusqu’à l’exécution d’un conteneur	54
Figure 15 Durée d’installation et de démarrage du conteneur après installation	55

Figure 16 Mémoire disponible sur le système	56
Figure 17 Planification du travail	64

Table des tableaux

Tableau 1 Liste des types de ressources disponibles	7
Tableau 2 Liste commandes disponible	8
Tableau 3 Comparaison des modèles d’orchestration centralisés et décentralisés	26
Tableau 4 Comparaison des solutions existantes	60

Table des extraits de code

Code 1 Configuration par défaut	6
Code 2 Configuration d’installation	9
Code 3 Configuration d’un runtime de conteneur	10
Code 4 Configuration d’un conteneur	11
Code 5 Configuration d’un réseau de conteneurs	11
Code 6 Commande d’installation	12
Code 7 Administration via Terraform	12
Code 8 Service gRPC pour l’orchestrateur	30
Code 9 Service gRPC pour les contrôleurs	31
Code 10 Structure interne de la file d’attente	32
Code 11 Encapsulation de la file d’attente pour les opérations d’écriture	33
Code 12 Encapsulation de la file d’attente pour un accès concurrent	33
Code 13 Structure d’une ressource transmise lors de la réconciliation	34
Code 14 Structure d’une ressource terminale	34
Code 15 Structure de réponse d’une réconciliation	35
Code 16 Aiguillage sur le schéma d’une ressource	37
Code 17 Extrait d’une liste d’attributs retournée pour un lien réseau	39
Code 18 Structure finale représentant l’état d’un lien réseau	39
Code 19 Structure intermédiaire générée par la macro Builder	40
Code 20 Extrait du constructeur de la ressource link	40
Code 21 Commande QEMU utilisée pour les mesures de performance	53

REMERCIEMENTS

Je tiens à remercier Monsieur Florent Glück, mon enseignant superviseur, pour son accompagnement tout au long du projet de semestre et du présent travail de bachelor.

OS POUR LE DÉPLOIEMENT DE SERVICES CONTENEURISÉS

ORIENTATION: INFORMATIQUE LOGICIELLE

Descriptif:

Le déploiement d'applications conteneurisées s'appuie désormais largement sur des approches déclaratives, en particulier dans les environnements clusterisés. En revanche, la configuration du système hôte reste souvent prise en charge par des outils impératifs distincts, ce qui introduit une rupture entre la gestion de l'infrastructure sous-jacente et celle des applications.

Cette séparation limite l'automatisation complète du déploiement et complique la reproductibilité, en particulier dans les contextes de déploiement unitaire, où les solutions conçues pour des environnements clusterisés peuvent s'avérer peu adaptées.

Le but de ce projet est donc de pallier à ce manque, en créant une distribution Linux légère destinée à la mise en place d'infrastructures conteneurisées. Ce système d'exploitation devra être léger en ressources, configurable via une API entièrement déclarative et sécurisé (conteneurs rootless, etc.).

Travail demandé:

- Système développé ne se basant pas sur une distribution Linux existante.
- Gestion de la configuration du système réalisée de manière déclarative.
- Gestion des ressources nécessaires à la conteneurisation (conteneurs, interfaces réseau Ethernet) via ces fichiers déclaratifs.
- Exécution des conteneurs de manière rootless afin d'augmenter la sécurité.
- Administration du système uniquement via une API, sans accès distant (SSH) ou interface procédurale.
- Test et validation de la solution réalisée sur des use-cases réels.
- Comparaison de la solution développée à Talos Linux et NixOS.
- Création d'un système permettant de facilement déployer/installer la solution sur machine physique ou infrastructure Cloud.

Candidat:
ARROYO FRÉDÉRIC

Filière d'études: ISC

Professeur responsable:
GLÜCK FLORENT

En collaboration avec: n/a

Travail de bachelor soumis à une
convention de stage en entreprise: non

Travail soumis à un contrat de
confidentialité: non

RÉSUMÉ

La conteneurisation s'est imposée comme le mode standard de déploiement des applications, avec un mode d'opération de plus en plus déclaratif. Les systèmes d'exploitation sous-jacents reposent toutefois majoritairement sur des distributions génériques, dont le modèle d'exécution demeure impératif et qui n'intègrent pas nativement la conteneurisation. Les solutions plus spécialisées existantes présentent des lacunes majeures dans le contexte de déploiements modestes ou embarqués. Ce travail présente la conception et l'implémentation d'un système d'exploitation minimaliste, dédié au déploiement déclaratif de conteneurs sur une machine unique. Le système repose sur un modèle de ressource homogène, décrivant conjointement la configuration de l'hôte et des conteneurs, dont l'état est maintenu par un orchestrateur centralisé au moyen d'une boucle de réconciliation continue. L'implémentation, réalisée en Rust sans se baser sur une distribution existante, comprend un processus d'initialisation, un environnement utilisateur restreint dépourvu de shell et d'accès interactif, ainsi qu'un ensemble de contrôleurs dédiés au réseau, aux conteneurs et à la configuration système. La validation du système, menée au moyen de tests unitaires, d'intégration et de bout en bout, démontre sa capacité à déployer une application complexe et à en préserver l'état à travers un redémarrage. Les mesures effectuées établissent un démarrage de conteneur en 5.1 secondes depuis une image ISO, sans installation préalable, ainsi qu'une exécution possible d'un serveur web avec seulement 160 MiB de mémoire vive, un seuil nettement inférieur à celui des solutions comparées. Ce travail conclut sur l'atteinte des objectifs techniques fixés, tout en identifiant des limites, notamment l'absence de journalisation et de supervision, ouvrant la voie à des développements futurs.



Candidat:
ARROYO FRÉDÉRIC

Filière d'études: ISC

Professeur responsable:
GLÜCK FLORENT

En collaboration avec: n/a

Travail de bachelor soumis à une
convention de stage en entreprise: non

Travail soumis à un contrat de
confidentialité: non

Glossaire

Capability: unité de privilège Linux, plus fine que la distinction binaire root/non-root, permettant d'accorder à un processus un sous-ensemble précis des privilèges normalement réservés à root (par exemple CAP_NET_ADMIN pour l'administration réseau)

Cgroup – Control Group: mécanisme du noyau Linux permettant de limiter, comptabiliser et isoler l'usage de ressources (CPU, mémoire, etc.) d'un ensemble de processus

MiB, GiB – Mébibyte, Gibibyte: unité de mesure en puissance de deux ($1 \text{ MiB} = 1024 * 1024 \text{ octets}$)

Mutex: verrou d'exclusion mutuel

Namespace: mécanisme du noyau Linux permettant d'isoler un processus d'une ressource système donnée (réseau, système de fichiers, identifiants de processus, etc.), de sorte que ce processus perçoive une instance propre de cette ressource

Rootless: mode d'exécution dans lequel un processus, tel qu'un runtime de conteneurs, fonctionne sans privilège root, réduisant ainsi l'impact d'une éventuelle compromission

Terraform: outil d'infrastructure-as-code permettant de décrire, au moyen de fichiers déclaratifs, l'état souhaité d'une infrastructure, puis de le réconcilier via des fournisseurs (*providers*) dédiés à chaque plateforme ou système ciblé

TPM 2.0 – Trusted Platform Module 2.0: Un TPM 2.0 est un composant de sécurité, typiquement matériel, implémentant la deuxième édition de la norme ISO/IEC 11889. Il permet de générer, de stocker et d'utiliser des clefs cryptographiques de manière protégée, et de mesurer l'état du système lors du démarrage [1]. La version 2.0 étend les algorithmes cryptographiques disponibles et introduit diverses améliorations par rapport aux versions précédentes.

Introduction

Contexte et problématique

La conteneurisation s'est imposée comme le mode standard de déploiement des applications dans les environnements modernes. Les outils qui gravitent autour d'elle, tels que Docker Compose [2] ou Kubernetes [3], adoptent un mode d'opération déclaratif dans lequel l'utilisateur décrit l'état désiré et le système se charge de le maintenir. Ce paradigme a également gagné l'infrastructure avec des outils tels que Terraform [4], permettant de gérer l'infrastructure sous forme de code (IaC) [5] plutôt qu'au moyen de procédures manuelles. Le modèle GitOps [6] pousse cette logique plus loin, en faisant d'un dépôt Git l'unique source de vérité de la configuration ainsi déclarée, toute modification de l'infrastructure ou des applications déployées transitant par un commit versionné dans ce dépôt plutôt que par une action directe sur le système cible. Il s'agit d'une partie importante de la mise en œuvre des pratiques DevOps et de CI/CD. Dans ces approches, l'interaction directe avec le système d'exploitation devient l'exception plutôt que la règle.

Toutefois, les systèmes d'exploitation sous-jacents reposent encore majoritairement sur des distributions génériques, administrées avec des outils de gestion de configuration dont le modèle d'exécution reste fondamentalement impératif, ce qui complique l'automatisation et la reproductibilité. De plus, ces distributions n'intègrent pas la conteneurisation comme un élément de première classe du système et nécessitent de configurer et gérer les conteneurs de manière distincte du système d'exploitation. De ce fait, la complexité de la configuration et du maintien en état repose entièrement sur l'administrateur.

Certaines solutions plus spécialisées existent, alliant déclarativité et support natif pour la conteneurisation, mais elles souffrent de lacunes diverses. NixOS [7] permet une gestion déclarative de l'ensemble du système, hôte et conteneurs, mais cette déclarativité reste ponctuelle : l'état désiré est appliqué une seule fois, à l'exécution d'une commande, sans qu'aucune boucle de contrôle ne surveille le système ni n'en corrige les dérives susceptibles de survenir par la suite. D'autres solutions, telles que Talos Linux [8], permettent une administration entièrement déclarative et continue, mais s'intègrent étroitement avec Kubernetes, au prix d'une complexité opérationnelle et d'une empreinte mémoire disproportionnée pour le simple déploiement de quelques conteneurs.

Il manque donc un système d'exploitation dans lequel il est possible de décrire la configuration de l'hôte et des conteneurs dans un modèle déclaratif unique et homogène, et capable de maintenir cet état de manière continue et autonome. Un tel système serait particulièrement pertinent pour les

déploiements modestes ou embarqués, où la complexité d'un orchestrateur complet n'est pas justifiée, mais où il est néanmoins souhaitable de disposer d'un système déclaratif.

Objectifs

L'objectif central est de fournir un système d'exploitation entièrement configurable selon un modèle déclaratif unique, dans lequel la configuration de l'hôte et celle des conteneurs sont intégrées de manière homogène. La conteneurisation est traitée comme un élément natif du système : les conteneurs sont décrits et gérés au même titre que les autres ressources, sans couches externes. En particulier, le système doit maintenir l'état désiré de manière continue et autonome, en s'appuyant sur une boucle de contrôle qui surveille en permanence l'état réel, détecte automatiquement les dérives et les corrige sans intervention humaine.

Parce que le système est destiné à des déploiements exposés à Internet et fonctionne sans surveillance constante, la sécurité est une exigence primordiale. Elle se concrétise par une isolation forte des composants système entre eux, une réduction maximale de la surface d'attaque et l'absence de tout accès interactif direct (notamment pas d'accès SSH). Le mode d'interaction entièrement déclaratif et automatisé permet de rendre cette absence d'accès direct viable.

Pour garantir une empreinte mémoire minimale et une maîtrise complète sur le fonctionnement du système, la solution est construite directement au-dessus du noyau Linux [9], sans recourir à une distribution préexistante. Elle comprend un processus d'initialisation (PID 1), un environnement utilisateur (user-space) restreint (sans shell ou systemd) qui gère le système, et intègre uniquement le runtime de conteneurs et un bootloader comme composants externes¹. Ce choix de conception permet de réduire la complexité superflue et de limiter encore la surface d'attaque.

Plusieurs modes de déploiement sont pris en charge par le système : une image ISO bootable, utilisable sans installation (mode éphémère), et permettant également l'installation de la solution de manière pérenne sur un disque. Le système doit également être aussi homogène que possible à travers des environnements physiques, virtualisés, embarqués, et cloud: une même configuration doit fonctionner sur ces environnements sans nécessiter de modifications majeures. Cela implique également le support des architectures Intel/AMD et ARM.

Cadre du travail et méthodologie

Ce travail s'inscrit dans l'obtention du titre de Bachelor of Science en Informatique et systèmes de communication, orientation Informatique logicielle, à la Haute école du paysage, d'ingénierie et d'architecture de Genève (HEPIA). La problématique abordée est issue d'une motivation personnelle, née de l'expérience dans l'administration de petites infrastructures, où le maintien manuel de systèmes

¹Par « composant externe », il est entendu ici un logiciel tiers fonctionnant comme un service ou un processus indépendant, par opposition aux bibliothèques logicielles (souvent appelés packages ou librairies).

hétérogènes et l'absence de correction automatique des dérives de configuration ont motivé la recherche d'une solution plus robuste. Réalisé à plein temps du 11 mai au 19 août 2026, il reflète l'état des connaissances et des technologies à cette période. Il a été précédé du projet de semestre, disponible sur https://gitedu.hesge.ch/flg_bachelors/ps/2025/container_os, qui a permis une analyse détaillée de la problématique, l'identification des briques techniques nécessaire et l'établissement des premiers éléments conceptuels et techniques. L'ensemble du code source produit est disponible sur le dépôt Git institutionnel à l'adresse suivante : https://gitedu.hesge.ch/flg_bachelors/tb/2026/container-infrastructure-deployment-os. L'état du dépôt au moment de la publication du présent document est disponible sur le tag `v0.0.0-dev.3.thesis` tandis que le dernier code testé est disponible sur la branche `main`.

Des intelligences artificielles (IA) génératives ont été utilisées de manière ponctuelle pour améliorer la qualité rédactionnelle un texte de base contenant l'intégralité du fond et de l'organisation a toujours été fourni en amont. Pour le code, elles ont principalement servi d'outil d'analyse, en complément des outils d'analyse de code traditionnels, afin de détecter d'éventuelles failles, bogues ou usages contraires aux bonnes pratiques. L'IA a également été employée pour conforter l'exhaustivité des recherches documentaires et des efforts de débogage : une fois la collecte humaine jugée suffisante ou qu'elle se heurtait à une impasse, l'IA a été interrogée pour signaler d'éventuels angles morts, en suggérant des mots-clés ou des références complémentaires. Ces suggestions ont toujours été vérifiées avant d'être utilisées. L'IA n'a explicitement pas été utilisée dans le cadre de la production de code, pour prendre des décisions architecturales ou organisationnelles relatives au code, ainsi que pour effectuer recherches et débogages initiaux. Un compte rendu détaillé et exhaustif de ces usages figure au sein de l'[annexe A, Utilisation de l'intelligence artificielle](#).

Le travail a été structuré en trois jalons principaux. Une première phase conceptuelle, jusqu'à la fin mai, a permis de poser les bases technologiques. Une phase de développement a suivi jusqu'à fin juin, intégrant l'ensemble des fonctionnalités strictement nécessaires à l'obtention d'une solution testable et comparable répondant aux objectifs techniques de l'énoncé. Enfin, l'implémentation de fonctionnalités supplémentaires et l'amélioration continue de la solution ont occupé la période allant jusqu'à fin juillet. Le déroulé exact des différentes étapes est disponible dans le [chapitre 6.2, Résultats académiques](#).

Structure du document

Le chapitre 1 présente le système du point de vue de l'utilisateur, sans détail d'implémentation. Le chapitre 2, consacré à la conception du système, expose les décisions architecturales conceptuelles majeures. Le chapitre 3 détaille ensuite les choix techniques retenus pour la réalisation de cette architecture, ainsi que les détails d'implémentation importants. Le chapitre 4 présente la démarche de validation adoptée, ainsi que l'analyse de performance, dont les résultats sont ensuite comparés au

chapitre 5 avec NixOS et Talos Linux. Le chapitre 6 dresse le bilan du travail réalisé et en discute les limites et perspectives.

Chapitre 1

Présentation du système

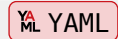
Ce chapitre présente le système du point de vue de l'utilisateur, sans entrer dans les détails d'implémentation, et illustre la manière dont les objectifs présentés dans l'introduction se traduisent concrètement dans son mode d'administration. Le modèle d'administration général est d'abord introduit, avant que le modèle de ressources, ainsi que les principes de déclarativité et d'immutabilité, ne soient détaillés. L'administration du système via l'API et son client en ligne de commande est ensuite présentée, suivie des mécanismes d'installation, de la gestion des conteneurs et des prérequis matériels. Le chapitre se conclut par un exemple d'utilisation combinant l'ensemble des éléments présentés.

1.1. Vue d'ensemble

Le système est administré intégralement via une API, à laquelle l'accès est possible soit directement au moyen d'un client en ligne de commande (CLI), soit via un provider Terraform, ce dernier étant, par nature, limité à la configuration du système. En particulier, l'administrateur n'a pas besoin de télécharger des paquets additionnels ou de configurer préalablement l'hôte avant de pouvoir l'administrer et l'utiliser pour y exécuter des conteneurs. L'ensemble de la configuration est décrit dans un unique fichier au format YAML, structuré sous la forme de plusieurs documents distincts, séparés par ---.

Une configuration minimale, définissant un accès à l'API sans authentification ainsi qu'une interface réseau configurée en DHCP, est présentée dans le code 1 :

```
1  ---
2  schema: api
3  auth: none
4  listen: 0.0.0.0
5  ---
6  schema: network:link
7  name: eth0
8  admin_up: true
9  ---
10 schema: network:dhcp
11 name: eth0
```

YAML

Code 1. – Configuration par défaut

Configuration par défaut lors du démarrage du système depuis l'image ISO.

Cette configuration, qui est la configuration par défaut lors du démarrage du système depuis une image ISO, permet de mettre en place le strict minimum pour que le système soit accessible : l'interface réseau est activée, une configuration réseau est réceptionnée par DHCP, et l'API est accessible sans authentification.

Chaque document de ce fichier correspond à la configuration d'une ressource. Au sein du système, une ressource représente un aspect administrable de celui-ci, tel qu'une interface réseau, un conteneur ou un client DHCP. L'utilisateur décrit l'état souhaité pour cet aspect du système, sans se préoccuper de la manière dont cet état est atteint et le système va automatiquement tenter de faire réconcilier son état vers l'état désiré.

1.2. Ressources

Les ressources sont regroupées par domaine fonctionnel, tel que `network`, `container` ou `system`, puis par type au sein de ce domaine, par exemple `network:link` pour le lien réseau ou `container:instance` pour une instance de conteneur. Les ressources peuvent aussi être nommées, cette propriété dépendant du type : une ressource unique et globale, telle que la configuration DNS, ne nécessite pas de nom, tandis qu'une ressource pouvant être instanciée plusieurs fois doit être nommée afin de pouvoir la distinguer des autres instances du même type. Lorsqu'il est nécessaire de faire référence à une ressource, il est possible d'utiliser la syntaxe `<type>/<nom>` ou simplement `<type>` dans le cas d'une ressource globale. Au sein d'un document, le type est représenté par le champ `schema` et, outre les éléments permettant d'identifier de manière unique une ressource, le document contient également la spécification de la ressource, aussi appelée l'état désiré. Cette spécification contient les paramètres propres à l'instance spécifique de la ressource, tels que l'image d'un conteneur, l'état d'une interface réseau, ou les paramètres d'authentification de l'API. Il existe ainsi quatorze ressources, brièvement décrites dans le tableau 1 :

Schéma	Description
api	Configuration de l'accès à l'API du système
install	Configuration de l'installation sur disque
system:etc	Configuration d'un fichier dans /etc/
network:dns	Configuration de la résolution DNS
network:interface	Configuration du réseau de manière simplifiée
network:dhcp	Configuration d'un client DHCP sur une interface
network:link	Configuration d'une interface réseau
network:address	Configuration d'une adresse réseau
network:route	Configuration d'une route réseau
network:ntp	Configuration du temps via NTP
container:runtime	Configuration d'un runtime de conteneurs
container:network	Configuration d'un réseau de conteneurs
container:volume	Configuration d'un volume de conteneurs
container:instance	Configuration d'une instance de conteneur

Tableau 1. – Liste des types de ressources disponibles

Ces ressources constituent l'ensemble des aspects configurables du système et permettent de disposer d'un hôte de conteneurisation à part entière, sans qu'aucun composant ou service supplémentaire ne soit requis en dehors de ceux décrits par ces quatorze types de ressources. Les domaines `network` et `container` couvrent respectivement l'ensemble de la configuration réseau et l'ensemble du cycle de vie des conteneurs, tandis que les domaines `api`, `install` et `system` couvrent l'administration du système lui-même.

1.3. Déclarativité et immutabilité

Le système tentant automatiquement de réconcilier son état avec celui décrit par l'utilisateur, il s'agit donc d'un système déclaratif. Le fichier de configuration constitue la seule source de vérité du système, toute autre modification étant généralement impossible. Le système peut néanmoins être amené à réagir à des événements particuliers, tels que la déconnexion d'un câble réseau ou l'arrêt inattendu du moteur de conteneurs. Étant donné que le système ne se base, à chaque réconciliation, que sur l'état désiré et sur l'état actuellement observé, sans tenir compte de la manière dont ce dernier a été atteint, toute divergence introduite par un tel événement est traitée comme n'importe quel autre écart à corriger, et le système est donc en mesure de gérer automatiquement ce type de situation en n'effectuant que le plus petit nombre d'actions nécessaires pour réconcilier ces deux états.

Le système reste, dans son fonctionnement normal, entièrement immuable : la configuration ne peut être modifiée autrement que par l'API, et les fichiers nécessaires au démarrage, tels que ceux du répertoire `/bin`, sont stockés et exposés de manière immuable lorsque le système est en cours d'exécution. À chaque redémarrage, le système se reconstruit entièrement à partir de ces seuls

fichiers de démarrage et de la configuration, garantissant un état initial déterministe. Ces contraintes garantissent que l'état du système reste en permanence traçable et reproductible à partir du seul fichier de configuration.

1.4. Administration du système et API

Le système étant entièrement administré au travers de l'API, il est nécessaire de fournir un client permettant d'interagir avec celle-ci. Ce client s'appelle `cos-cli` et permet d'effectuer les actions essentielles sur le système à travers diverses commandes décrites dans le tableau 2 :

Commande	Description
<code>config validate <PATH></code>	Valide la configuration auprès du serveur sans l'appliquer
<code>config push <PATH></code>	Transmet le fichier de configuration au serveur
<code>config pull</code>	Télécharge le fichier de configuration actuel du serveur
<code>fs write <LOCAL_PATH> <VOLUME>:<PATH></code>	Écrit un fichier sur un volume de conteneur
<code>fs list [VOLUME:]<PATH></code>	Liste les fichiers, tant sur un volume que sur le système de fichier racine
<code>fs read [VOLUME:]<PATH></code>	Télécharge un fichier se trouvant sur le serveur
<code>resources list [SCHEMA]</code>	Liste les ressources, éventuellement filtré par type
<code>resources get <SCHEMA> [NAME]</code>	Consulte l'état d'une ressource
<code>system reboot</code>	Redémarre ou éteint le système

Tableau 2. – Liste commandes disponible

La syntaxe est `subsystem command <MANDATORY> [OPTIONAL]`.

Ce client permet notamment de transmettre une nouvelle configuration au système, de consulter l'état des ressources existantes, ou d'interagir directement avec les volumes des conteneurs, sans que l'utilisateur ait à se connecter directement à la machine hôte.

L'accès à l'API est lui-même régi par une ressource dédiée, au même titre que toute autre ressource du système, comme illustré dans le code 1. Cette ressource permet de définir le mécanisme d'authentification requis, ainsi que l'adresse depuis laquelle l'API est accessible, restreignant ainsi la surface d'exposition du système sur le réseau. L'absence d'authentification, utilisée dans la configuration par défaut, est destinée à un usage temporaire : la phase d'installation étant relativement courte, et le serveur ne possédant pas encore de données sensibles, il est considéré que laisser un

tel accès ouvert durant quelques secondes (entre le démarrage de l'API et la transmission d'une configuration initiale) est convenable².

Lorsque la configuration est mise à jour, celle-ci n'est pas immédiatement acceptée : le serveur va d'abord valider que celle-ci soit syntaxiquement correcte et, dans la mesure du possible, que les données s'y trouvant sont valides. En cas d'erreur, même partielle, un message avec les détails sera retourné à l'utilisateur. De fait, seule une configuration entièrement correcte peut être transmise au serveur.

1.5. Installation du système et modes de fonctionnement

L'installation du système s'effectue de la même manière que l'administration des ressources courante : en ajoutant un document d'installation dans la configuration. Ce document est présenté dans le code 2 :

```
1  ---
2  schema: install
3  disks:
4    - dev: /dev/vda
5      partitions:
6        - type: boot
7        - type: config
8          encryption:
9            provider: static
10             key: this-is-a-very-secure-password
11             autounlock: true
12        - type: data
13          encryption:
14            provider: tpm2
```

Code 2. – Configuration d'installation

Installation du système entièrement sur le disque /dev/vda.

Ce document d'installation décrit les trois volumes de stockages sur lesquels le système se repose : le disque de démarrage, contenant les divers artefacts, tels que le noyau, les binaires du système, et le bootloader. Ensuite, le volume contenant la configuration du système, puis un volume de donnée dans lequel seront stockées les images et les volumes des conteneurs.

Seul le disque de boot est obligatoire : omettre le disque de configuration et le disque de données permet d'avoir un système entièrement éphémère. Chaque redémarrage fournit un système complètement neuf, ce qui peut s'avérer particulièrement pratique dans le cadre de tests.

²C'est aussi le parti pris d'autres solutions, tel que Talos Linux.

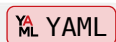
Les disques supportent le chiffrement à travers un TPM 2.0, ou à travers une clef statique. Dans le cas de la clef, celle-ci peut optionnellement être stockée conjointement avec la configuration, au détriment de la sécurité, mais permettant de déverrouiller le volume de manière autonome.

Il est aussi possible d'omettre complètement le document d'installation, auquel cas rien ne sera sauvegardé, et le système sera restitué dans son état d'origine après un redémarrage. Dans ce mode de fonctionnement, le système demeure entièrement utilisable, y compris les volumes de conteneurs, qui seront stockés en mémoire.

1.6. Conteneurs

La conteneurisation repose sur le concept de runtime, qui est simplement l'interface permettant d'exécuter des conteneurs. Chaque conteneur est associé à un runtime qui sera en charge de l'administrer. Actuellement, seul Podman est disponible comme runtime, mais il est toujours utile de pouvoir instancier plusieurs fois ce runtime, par exemple lorsqu'il est souhaitable de disposer de container entièrement « rootless ». Le code 3 décrit une telle configuration :

```
1  ---
2  schema: container:runtime
3  name: rootless
4  engine: podman
5  uid: 1000
6  gid: 1000
7  depends_on:
8    - network:dns
9    - network:route/eth0-dhcp
```



Code 3. – Configuration d'un runtime de conteneur

Un runtime de conteneur utilisant Podman est configuré en mode « rootless ».

Le code 3 spécifie l'id utilisateur et groupe non-root, mais aussi un ensemble de dépendances à des ressources DNS et réseau. Ces dépendances ne sont pas strictement nécessaires, mais permettent d'éviter d'instancier le runtime tant que la couche réseau n'est pas disponible, ce qui pourrait avoir pour effet d'empêcher le téléchargement d'images de conteneurs, et créerait des messages d'erreur temporaire. La nature déclarative du système décrite dans le [chapitre 1.3, Déclarativité et immutabilité](#) rend toutefois ces dépendances optionnelles, car il est capable de se rétablir seul.

Une fois un runtime configuré, il est possible d'y créer diverses ressources, telles que des réseaux de conteneurs ou des instances de conteneurs, comme décrit dans le code 4 :

```
1 ---
2 schema: container:instance
3 name: demo
4 image: docker.io/alpine/curl:latest
5 restart: never
6 runtime: rootless
7 cmd: [http://10.0.2.2:1234]
```

YAML

Code 4. – Configuration d'un conteneur

Configuration d'un conteneur qui va requêter une fois un serveur HTTP puis s'arrêter.

Le code 4 crée une instance de conteneur nommé « demo » et ayant pour image la dernière version de cURL [10] sur Alpine Linux [11]. Cette instance va être démarrée, elle va requêter une URL, et si cela réussit, alors le conteneur s'arrête. Il est aussi possible de supporter des cas plus complexes grâce aux réseaux de conteneurs et à la publication de ports sur l'hôte via une syntaxe similaire à Docker Compose. La création de réseau de conteneurs se fait via une ressource dédiée, illustrée dans le code 5 :

```
1 ---
2 schema: container:network
3 name: my-network
4 runtime: rootless
```

YAML

Code 5. – Configuration d'un réseau de conteneurs

Configuration d'un réseau de conteneur sur le runtime de conteneur nommé « rootless ».

Tout comme pour la création de conteneurs, il est nécessaire de spécifier le runtime sur lequel sera créé ce réseau, et il est naturellement impossible d'y ajouter des conteneurs extérieurs à ce runtime sur le réseau ainsi créé.

1.7. Prérequis matériels

Le système nécessite très peu de ressources : sur un processeur 64 bits, il est possible de démarrer un serveur web minimal avec seulement 160 MiB de mémoire vive. Il est même possible de démarrer le système sur moins de 80 MiB de mémoire vive si les aspects liés à la conteneurisation ne sont pas nécessaires, par exemple, afin de fournir un routeur rudimentaire. En outre, dans le cadre d'une installation complète, le système ne nécessite que 1 GiB de stockage. L'ensemble de ces chiffres sont détaillés dans le [chapitre 4.3, Analyse de performance](#).

1.8. Exemple d'utilisation

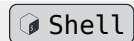
Le scénario suivant illustre l'ensemble des éléments présentés dans ce chapitre : une instance de machine virtuelle, fraîchement créée chez un fournisseur cloud et démarrée depuis l'image ISO du

système, est configurée, installée sur son disque, puis déployée avec un serveur HTTP retournant le message « Hello, world! ».

L'ensemble des ressources nécessaires à ce scénario est combiné au sein d'un unique fichier de configuration, versionnable dans un dépôt Git. Ce fichier doit se situer sur la machine depuis laquelle le client `cos-cli` est exécuté. L'annexe B contient la configuration complète utilisée dans cet exemple, combinant l'installation du système sur disque, l'accès à l'API, la configuration réseau, ainsi qu'un conteneur exécutant un serveur HTTP minimal et accessible depuis l'extérieur.

Une fois la machine virtuelle démarrée depuis l'image ISO, la commande présentée au code 6 permet de transmettre cette configuration au serveur dont l'adresse est désignée par `$IP` elle déclenche à la fois l'installation du système sur le disque et le déploiement du serveur HTTP :

```
1 $ cos-cli --server $IP config push ./config.yaml
```



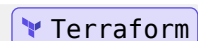
Code 6. – Commande d'installation

La commande est exécutée depuis une machine distincte du serveur cible, disposant d'un accès réseau à celui-ci.

Le serveur HTTP est accessible depuis l'adresse `$IP` et le port 8080 de l'instance moins de 30 secondes après l'exécution de la commande précédente, cette durée étant mesurée et détaillée au [chapitre 4.3.1, Rapidité](#). Ce temps inclut l'installation complète du système sur le disque, le redémarrage de la machine, et le téléchargement de l'image. Aucune configuration supplémentaire n'est nécessaire sur le fournisseur cloud au-delà de l'ouverture du port correspondant. La même commande est réutilisée pour toute mise à jour ultérieure de la configuration, par exemple pour modifier l'image du conteneur ou ajouter un second service. Le système se charge de réconcilier automatiquement l'état effectif avec la nouvelle configuration transmise, sans redémarrage ni interruption des ressources déjà en place, tel que décrit dans le [chapitre 1.3, Déclarativité et immutabilité](#).

Le même processus est également accessible via Terraform, dans le cas où l'instance cloud et le déploiement du système sont administrés au sein d'une même infrastructure :

```
1 resource "containeros_config_push" "my_server" {  
2     server = some_cloud_provider.vm.ip  
3     config = file("./config.yaml")  
4 }
```



Code 7. – Administration via Terraform

Provisionnement et configuration de la machine au sein d'un même plan Terraform.

Cette même configuration fonctionne indifféremment sur une instance cloud, comme illustré, sur un serveur bare metal ou sur une plateforme embarquée telle qu'un Raspberry Pi, seul le chemin vers le

disque de stockage étant propre à chaque serveur (par exemple, `/dev/sda` pour un disque SATA, `/dev/vda` pour un disque VirtIO, etc.).

Chapitre 2

Conception du système

Ce chapitre présente l'architecture conceptuelle du système, en explicitant les décisions architecturales importantes. Les composants fondamentaux du système sont d'abord brièvement introduits, avant que le modèle de ressource ne soit détaillé, notamment sa structure, les relations qui lient les ressources entre elles, ainsi que les différentes catégories. L'orchestrateur et le mécanisme de réconciliation sont ensuite présentés, en couvrant le rôle du contrôleur, le choix d'une architecture d'orchestration centralisée, la planification des cycles de réconciliation, la suppression des ressources, ainsi que la réaction aux événements internes et externes qui peuvent survenir en dehors de ce cycle.

2.1. Composants du système

Le système repose sur quatre notions fondamentales, détaillées dans le reste de ce chapitre. Une ressource représente un élément administrable du système, décrit par une spécification correspondant à l'état désiré. L'orchestrateur est le composant central chargé de stocker l'ensemble des ressources, d'ordonnancer leur réconciliation et de maintenir la cohérence des relations qui les lient. Un contrôleur implémente, pour un ou plusieurs types de ressources, la logique permettant de faire converger l'état observé vers la spécification. Enfin, la réconciliation désigne le processus, exécuté de manière continue, par lequel un contrôleur observe l'état physique réel d'une ressource, le compare à sa spécification, et produit les actions correctives nécessaires à leur convergence. La figure 1 illustre la manière dont ces composants interagissent entre eux :

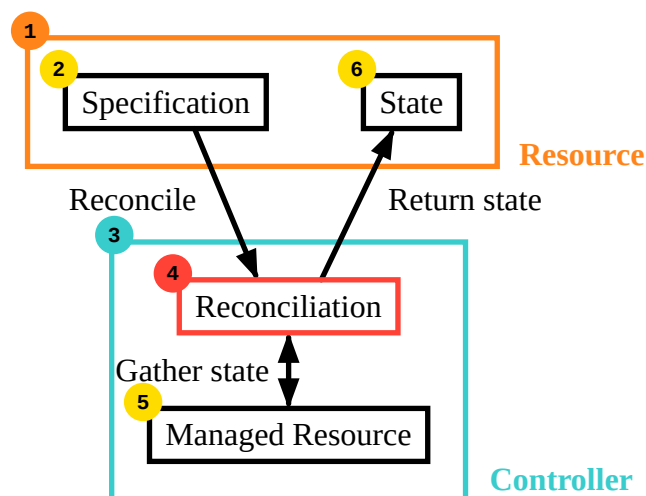


Figure 1. – Réconciliation d'une ressource

Le système observe l'état actuel de la ressource, calcule l'écart avec la spécification, et applique les actions correctives.

Lorsque l'orchestrateur souhaite réconcilier une ressource **1**, il va transmettre la spécification **2** au contrôleur responsable **3**. Durant la procédure de réconciliation de ce dernier **4**, l'état réel de la ressource concernée va être récupéré **5** et les actions nécessaires afin de faire converger cet état avec la spécification vont être entreprises. Une fois cela fait, un nouvel état est retourné pour être enregistré dans la ressource **6**.

2.2. Modèle de ressource

2.2.1. Structure d'une ressource

Toute ressource partage une structure commune composée d'un type, d'un nom, d'une spécification, d'un état observé représentant l'état réel à un instant T , d'un statut et d'une phase. Le type fixe le schéma structurel de la spécification et de l'état observé, tandis que le nom identifie de manière unique une instance au sein de ce type. La spécification constitue l'état désiré, tandis que l'état observé reflète la réalité constatée par le contrôleur l'état observé est réévalué à chaque cycle de réconciliation, tel que détaillé dans le [chapitre 2.3.2, Contrôleur et déclarativité](#). Entre deux réconciliations, l'état observé demeure celui résultant de la dernière réconciliation effectuée. Le type et le nom d'une ressource sont immuables : modifier l'un ou l'autre de ces champs équivaut à supprimer la ressource existante et à en recréer une nouvelle.

2.2.2. Relations entre les ressources

Deux types de relations lient les ressources entre elles : la possession et la dépendance. Une ressource peut avoir au plus un détenteur celui-ci dispose des droits de modification et de suppression sur la ressource, ainsi que le droit de consulter la spécification et l'état observé de celle-ci. Le détenteur d'une ressource donnée est simplement celui qui crée cette ressource. La dépendance permet de consulter la spécification et l'état observé, sans pour autant pouvoir modifier la ressource.

Le lien de dépendance a aussi un impact sur la planification : lorsqu'une ressource A dépend d'une ressource B , il est raisonnable de ne pas tenter de réconcilier A tant que B n'est pas prête. En outre, ces relations ne sont jamais transitives : si A possède B , et B possède C , A ne possède pas C . Le mécanisme exact est décrit dans le [chapitre 2.3.4, Boucle de réconciliation](#).

2.2.3. Catégories de ressources

Les ressources sont classées en deux catégories primaires : les ressources qui peuvent être détenues, et celles ne pouvant pas l'être. Il s'agit respectivement de ressources privées et de ressources mutualisées.

Une ressource privée dispose d'un détenteur unique, disposant seul des droits de modification et de suppression sur celle-ci. Trois sous-catégories de ressources privées existent, selon la nature de ce détenteur et la persistance de la ressource à travers les redémarrages. Une ressource statique est détenue conceptuellement par l'administrateur système et ne peut être modifiée qu'en transmettant une nouvelle version de la configuration du système. Une ressource dynamique est détenue par une autre ressource, qui décide seule de son contenu et de son cycle de vie. Une ressource éphémère est

détenue par une autre ressource au même titre qu’une ressource dynamique, mais ne persiste jamais à travers un redémarrage du système, indépendamment de la persistance de la configuration elle-même cette dernière catégorie est notamment utilisée pour les ressources d’adresse ou de route obtenues dynamiquement par DHCP et qui, selon la spécification du protocole, ne devraient pas persister entre deux redémarrages [12].

Une ressource partagée, à l’inverse, ne dispose d’aucun détenteur propre : elle est détenue par l’orchestrateur, identifiée uniquement par son type et son nom. Son fonctionnement précis, en particulier la motivation derrière son existence, est détaillé dans le [chapitre 2.2.4, Ressources mutualisées](#). Il est à noter qu’une ressource mutualisée peut, au même titre qu’une ressource privée, être elle-même détentrice d’autres ressources : la distinction entre ces deux catégories ne porte que sur l’impossibilité d’être détenue, non sur la capacité à détenir.

Il convient de préciser que le qualificatif « privée » ne signifie pas que la ressource est invisible au reste du système : sa spécification et son état observé demeurent consultables par toute ressource déclarant une dépendance à son égard, conformément au [chapitre 2.2.1, Structure d’une ressource](#). Seuls les droits de modification et de suppression sont restreints à son détenteur.

Les ressources statiques sont les seules ressources que l’administrateur peut modifier. En effet, le lien de possession n’étant pas transitif, lorsque l’administrateur crée une ressource A qui crée à son tour une ressource B , le seul moyen pour l’administrateur d’agir sur la ressource B est de passer par A . Si l’utilisateur souhaite pouvoir interagir directement avec B , il doit alors faire en sorte que celle-ci ne soit plus gérée par A , et ensuite de la déclarer lui-même dans la configuration, ce qui rendrait cette ressource statique. Cette distinction entre statique et dynamique permet de restreindre l’accès de l’administrateur aux seules ressources pertinentes du point de vue de la configuration, en masquant les détails d’implémentation qu’une ressource statique délègue à ses enfants dynamiques. Cela rend aussi possible d’abstraire une configuration complexe, nécessitant plusieurs ressources, derrière une seule ressource simple, tel qu’illustré dans la figure 2 :

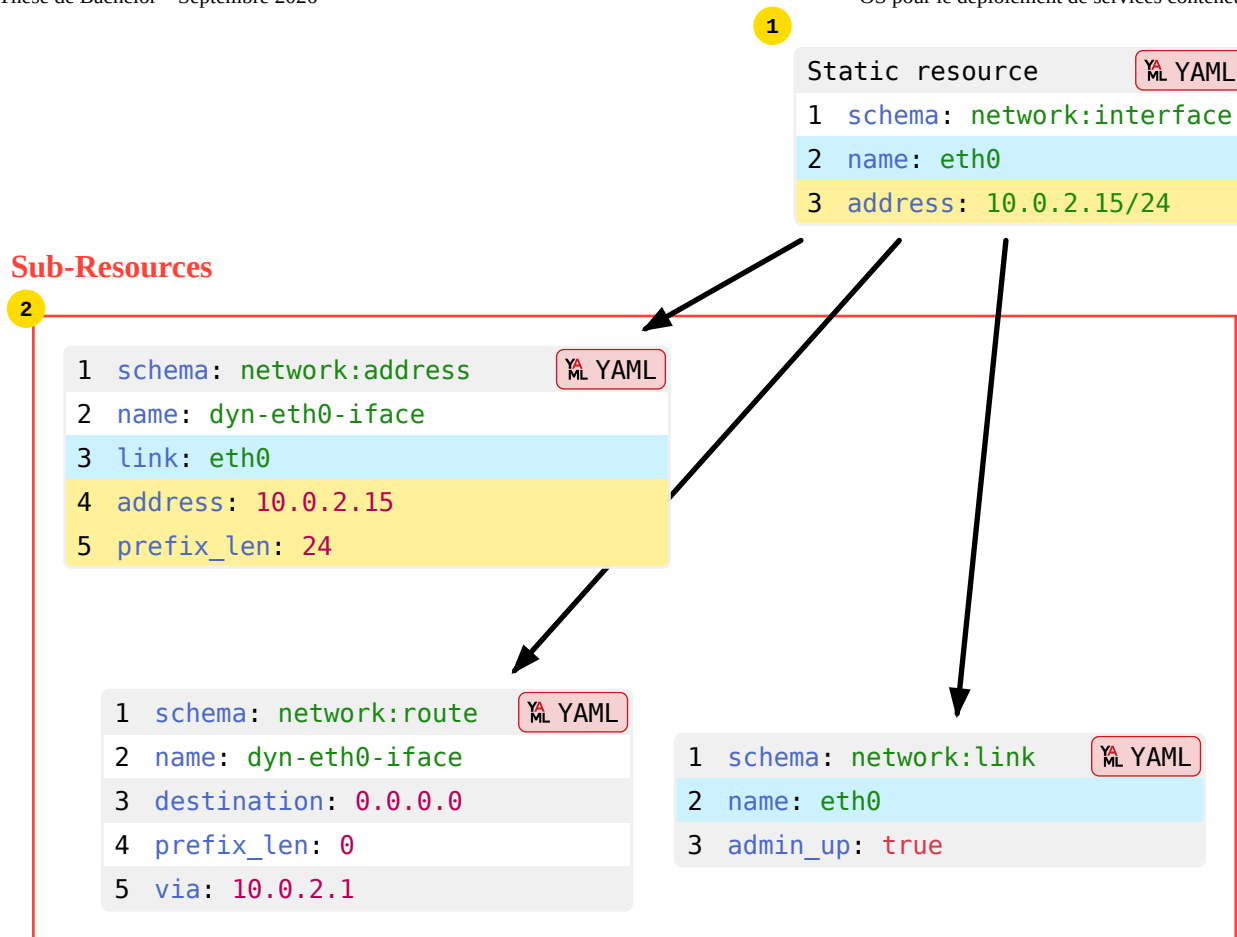


Figure 2. – Création de sous-ressources à partir d'une configuration utilisateur

Il est ainsi commun dans les systèmes existants (netplan, systemd-networkd, notamment) de configurer conjointement le lien réseau, les routes et les adresses associées au sein d'une seule et même configuration. Dans le présent système, une configuration de type `network:interface` **1** abstrait cette complexité en créant et en possédant plusieurs sous-ressources dynamiques (`network:link`, `network:address` et `network:route`), chacune correspondant à un aspect distinct **2**. Le détenteur gère entièrement la spécification de ces enfants : lorsque sa propre spécification est mise à jour puis réconciliée, alors celui-ci mettra à jour la spécification de ses enfants. L'administrateur du système n'a qu'à se préoccuper de la spécification de la ressource parente. D'autre part, cela permet de modéliser des aspects dynamiques par nature, tel que la configuration du réseau via DHCP, tel qu'illustré dans la figure 3 :

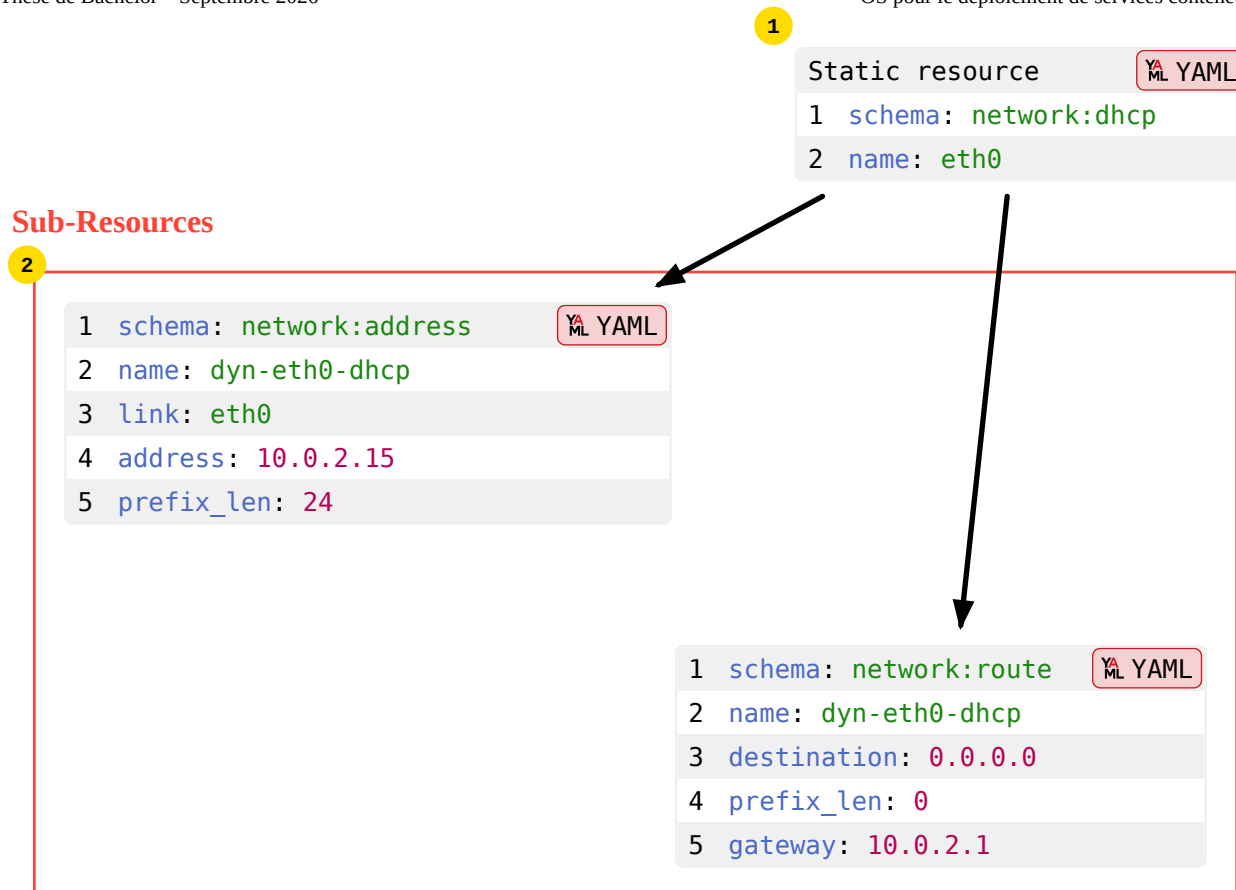


Figure 3. – Création de sous-ressources à partir d'une configuration DHCP

Une ressource de type `network:dhcp` est créée par l'utilisateur ① et, en arrière-plan, un client DHCP va configurer l'adresse et la route réseau lorsque celles-ci deviendront disponibles à travers le protocole ②. Dans ce cas, si l'administrateur avait la capacité de modifier ces sous-ressources, cela irait à l'encontre du fonctionnement nominal du DHCP.

2.2.4. Ressources mutualisées

Le partage d'une ressource est donc naturellement réalisé par un lien de dépendance. Toutefois, cela présuppose que la ressource soit créée par une autre ressource ou par l'administrateur du système. En raison d'usages et d'habitude commune adoptée au sein de certains logiciels, certaines ressources sont implicites à un point où les déclarer manuellement serait contre-productif. Toutefois, ne pas les déclarer explicitement pose le problème de l'attribution de la responsabilité de les concevoir et de les supprimer. C'est notamment le cas des images de conteneurs, comme illustré dans la figure 4 :

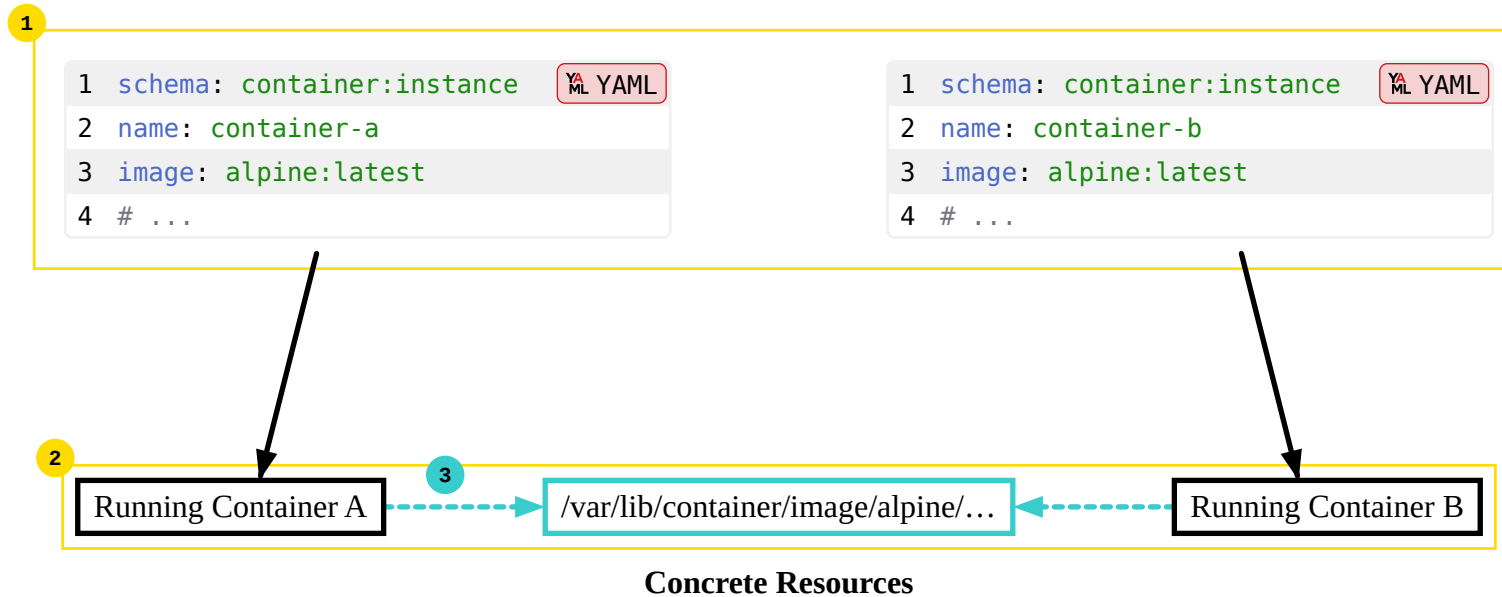


Figure 4. – Problème des ressources partagées implicitement

Illustre comment une ressource système va interagir avec une ressource concrète, et comment deux ressources concrètes différentes peuvent finalement interagir avec une seule et même autre ressource concrète.

Par exemple, deux configurations de conteneurs ① indépendantes créent deux conteneurs qui semblent eux aussi indépendants ②, mais qui pointent en réalité vers une même ressource, en l'espèce l'image ③. En raison des propriétés de la gestion des images dans un runtime de conteneurs, la création ne pose pas de problèmes l'image sera téléchargée par le runtime, puis utilisée par les conteneurs. En revanche, la suppression pose un problème : un runtime ne supprime jamais les images de lui-même, et, au fil de l'utilisation du système, les images prendraient de plus en plus de places sans jamais être supprimées.

Il n'est aussi pas possible de supprimer la ressource dès qu'un seul conteneur est supprimé : l'image est toujours requise par l'autre configuration. Selon la nature de la ressource sous-jacente, deux comportements seraient alors possibles : soit la suppression échoue et la réconciliation reste bloquée, soit la ressource est détruite et doit être recréé par l'autre configuration, ce qui est inefficace. La création d'une sous-ressource image par chaque conteneur est également impossible, toute ressource ne pouvant être possédée que par un seul parent.

C'est précisément ce problème que la catégorie des ressources mutualisées, introduite au [chapitre 2.2.3, Catégories de ressources](#), permet de résoudre. Contrairement à une ressource privée, une ressource mutualisée ne possède aucune spécification propre : l'orchestrateur en est l'unique détenteur et, en l'absence de tout paramètre, ne gère que sa création implicite et sa suppression. Il suffit qu'une ressource déclare une dépendance vers une ressource mutualisée pour que l'orchestrateur la crée si elle n'existe pas déjà si une autre ressource déclare ultérieurement dépendre de cette même ressource mutualisée, aucune seconde instance n'est créée, la dépendance référençant simplement l'instance existante.

L'orchestrateur décidant seul de la suppression d'une ressource mutualisée, la règle appliquée est simple : dès qu'aucune ressource ne dépend plus d'elle, elle est placée en cours de suppression, et suit alors les mêmes règles que toute autre ressource, détaillées au [chapitre 2.3.5, Suppression des ressources](#). De même, les principes de déclarativité présentés au [chapitre 2.3.2, Contrôleur et déclarativité](#) s'appliquent également à cette catégorie : en particulier, même en l'absence de spécification propre, une ressource mutualisée peut créer des sous-ressources dont le contenu est arbitraire.

La figure 5 illustre la résolution du conflit précédent grâce à une ressource mutualisée :

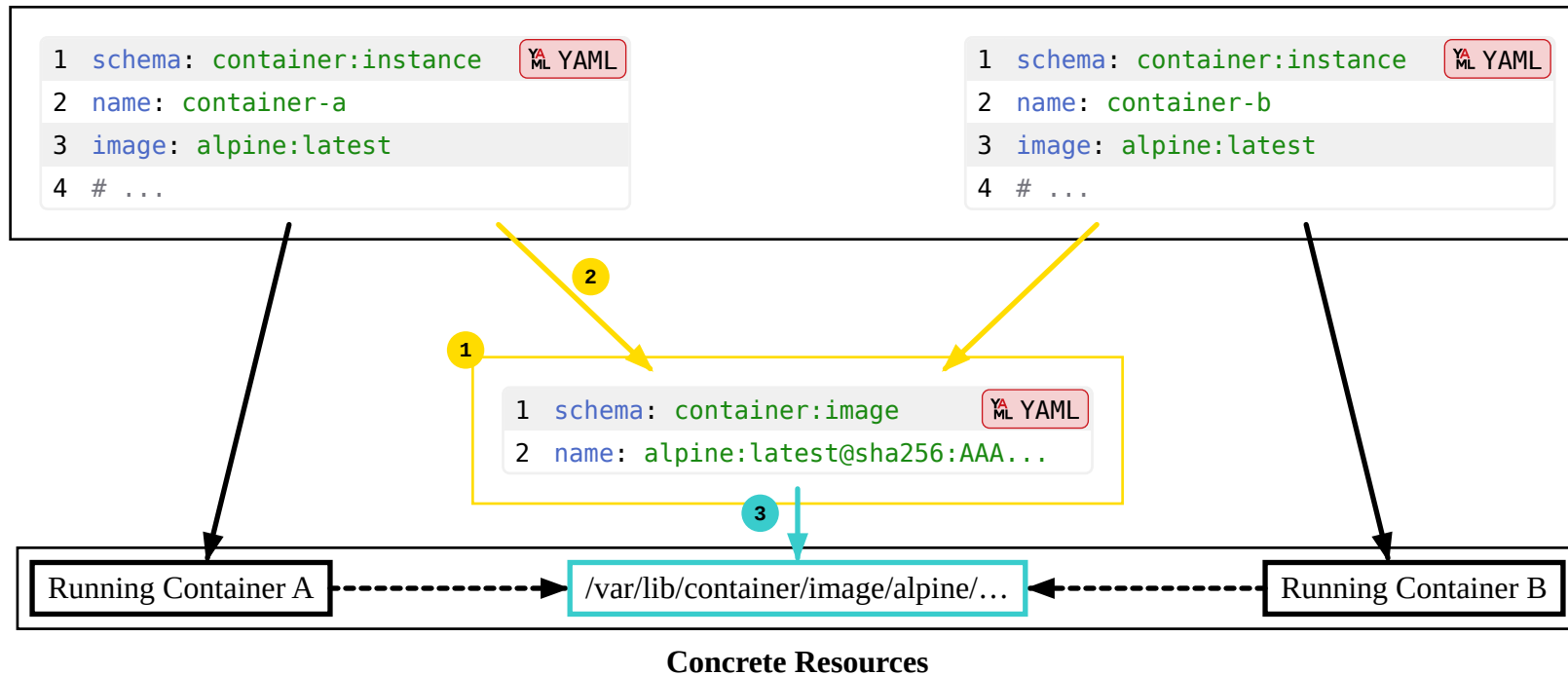


Figure 5. – Solution au problème du partage de ressources implicite

Illustre comment, en ajoutant une abstraction, la problématique de deux ressources concrètes indépendantes interagissant avec une troisième ressource concrète peut être résolue.

Dans la figure 5, les deux conteneurs déclarent chacun une dépendance envers la même ressource mutualisée d'image **2**, plutôt que de créer chacun leur propre ressource. Cette ressource mutualisée unique **1** est ainsi seule responsable de la ressource physique correspondante **3**. Il n'existe donc plus de conflit : si l'une des deux configurations change le nom de l'image référencée, cela crée simplement une nouvelle ressource mutualisée distincte, plutôt qu'un conflit sur l'ancienne.

Dans l'exemple des conteneurs, la réconciliation se déroule ainsi :

1. la ressource représentant la configuration du conteneur *A* récupère d'abord, d'une manière ou d'une autre, le nom complet de l'image configurée, avec son hash
2. à partir de cette information, elle déclare une dépendance envers une ressource mutualisée de type `container:image`, portant pour nom le nom complet de l'image avec son hash
3. cette ressource mutualisée se charge alors de télécharger l'image sur le disque cette opération, pouvant nécessiter plusieurs itérations, est effectuée en arrière-plan, la ressource conservant entre-temps le statut « pas prêt »
4. tant que la ressource `container:image` n'a pas atteint le statut « terminé », la ressource `container:instance` correspondante n'entreprend aucune autre action
5. une fois l'image téléchargée, la ressource `container:image` passe au statut « terminé »
6. la ressource `container:instance` crée alors le conteneur
7. le cycle de vie normal de la ressource se poursuit ensuite
8. lorsqu'un conteneur dépendant de cette image est supprimé, le lien de dépendance correspondant est rompu
9. si le nombre de dépendances entrantes de la ressource `container:image` tombe à zéro, l'orchestrateur ordonne sa suppression, ce qui efface l'image du disque
10. si d'autres conteneurs dépendent encore de cette même image, aucune action supplémentaire n'est entreprise.

2.3. Orchestrateur et réconciliation

2.3.1. Orchestrateur

L'orchestrateur est simplement le composant responsable de stocker l'ensemble des ressources du système, de s'assurer que les réconciliations ont lieu dans le bon ordre, et de garantir que le système se comporte correctement.

2.3.2. Contrôleur et déclarativité

La réconciliation est le processus qui assure que l'état observé d'une ressource converge vers sa spécification le contrôleur est le composant logiciel qui en a la responsabilité. Ce processus s'inscrit dans une boucle permettant de corriger toute dérive : à chaque itération, le contrôleur observe l'état physique réel de la ressource, le compare à sa spécification, puis produit les actions correctives nécessaires. La figure 6 illustre cette boucle :

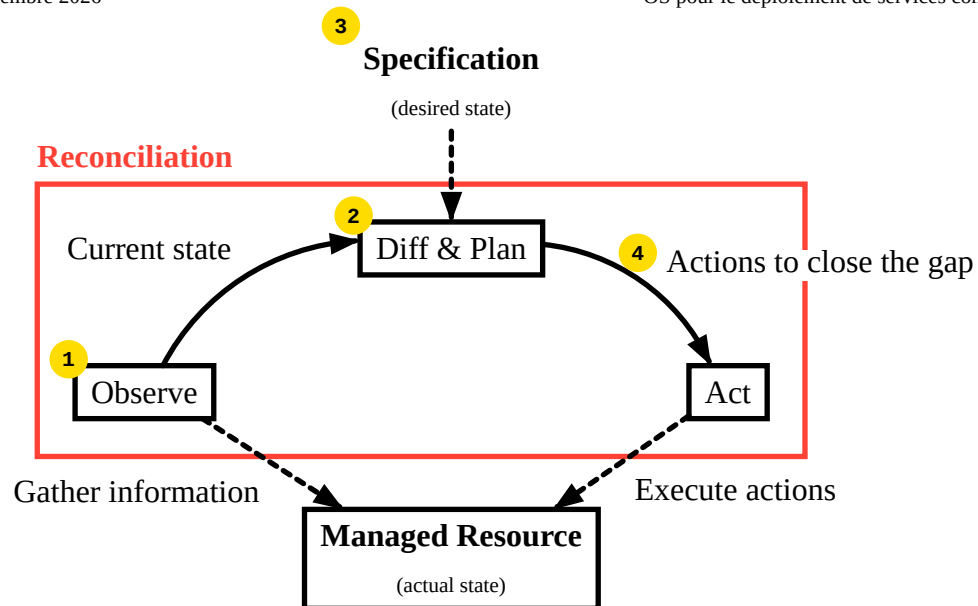


Figure 6. – Réconciliation d'une ressource

Le système observe l'état actuel de la ressource, calcule l'écart avec la spécification, et applique les actions correctives.

Dans le cas de la configuration d'un lien réseau³, par exemple, la spécification 3 pourrait indiquer que le statut administratif doit être « up »⁴. L'état physique de l'interface (statut administratif, statut opérationnel, adresse MAC, etc.) est d'abord récupéré 1 constituant ainsi l'état observé, puis comparé à la spécification 2. Dans le cas où le statut administratif serait « down », le système s'en rend compte et sait qu'il doit exécuter l'équivalent de `ip link set up` afin de mettre en route l'interface 4. Ce même mécanisme s'applique à toute modification ou suppression de la ressource : une modification de la configuration déclarative se traduit automatiquement par les actions correctives adéquates.

En ce qui concerne les sous-ressources, lors de la réconciliation d'une ressource donnée, celle-ci retourne d'une part le nouvel état observé, mais elle déclarera aussi l'ensemble de ses enfants avec leur spécification. La ressource est libre de déterminer comme elle le souhaite le contenu de la spécification de ces enfants spécification, la seule contrainte étant la validité vis-à-vis du schéma du type de ressource. De fait, même en l'absence de tout paramètre, comme dans une ressource mutualisée, rien n'empêche, lors de la réconciliation, de créer des sous-ressources avec une spécification arbitraire. En outre, supprimer un enfant de cette déclaration revient à demander la suppression de celui-ci.

2.3.3. Orchestration de la réconciliation

Le système est composé de plusieurs ressources, chacune gérée par une logique de réconciliation indépendante. Une réconciliation peut survenir de manière périodique ou en réponse à un événement, qu'il soit interne, tel qu'une mise à jour de la configuration, ou externe, comme un conteneur qui

³Un lien réseau correspond à ce qui est communément appelé une interface réseau.

⁴Linux fait la distinction entre l'état administratif et l'état opérationnel d'une interface. L'état administratif indique si l'administrateur souhaite utiliser le périphérique pour le trafic. L'état opérationnel indique la capacité d'une interface à transmettre ces données utilisateur [13].

s'arrête. À ce titre, deux modèles d'orchestration sont envisageables : un modèle centralisé, dans lequel une boucle unique appelle, à chaque itération, la méthode de réconciliation d'une ressource donnée et un autre modèle décentralisé, dans lequel chaque contrôleur contient sa propre boucle, organisée de la manière qui lui convient (une boucle par ressource, une boucle par type, etc.). Le modèle centralisé est illustré dans la figure 7 :

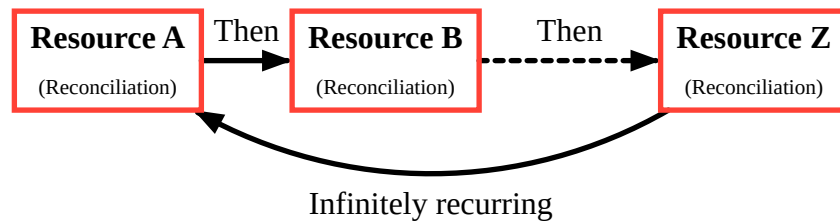


Figure 7. – Orchestration de la réconciliation dans un modèle centralisé

Illustre la manière donc la réconciliation individuelle de chaque ressource est intégrée vis-à-vis des autres ressources au sein d'un modèle de réconciliation centralisé.

L'orchestrateur exécute une unique boucle : à chaque itération, il retire de la file d'attente la ressource dont l'échéance de réconciliation est la plus proche, délègue sa réconciliation au contrôleur correspondant, traite la réponse de celui-ci (nouvel état observé, statut et sous-ressources déclarées), puis replanifie la ressource selon le délai retourné. Dans ce modèle, le contrôleur n'a pas connaissance des autres ressources, même celle sous son contrôle. La réconciliation se repose uniquement sur les informations de la ressource courante et d'éventuels détails sur d'autres ressources transmis par l'orchestrateur sur la base des liens de possession ou de dépendance.

Le modèle décentralisé adopte l'approche opposée en déplaçant la boucle dans chaque contrôleur, tel qu'illustré dans la figure 8 :

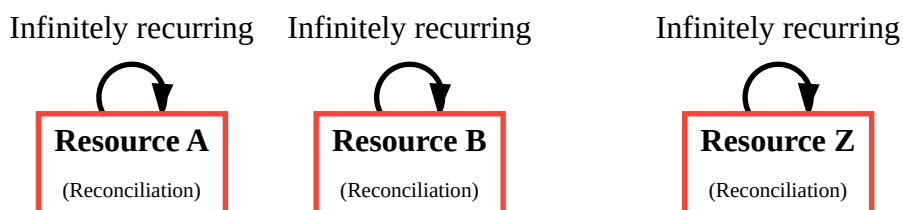


Figure 8. – Orchestration de la réconciliation dans un modèle décentralisé

Illustre la manière donc la réconciliation individuelle de chaque ressource est intégrée vis-à-vis des autres ressources au sein d'un modèle de réconciliation décentralisé.

Chaque contrôleur exécute sa propre boucle de réconciliation, indépendante de celle des autres contrôleurs, et détermine lui-même l'instant de la prochaine réconciliation de chacune des ressources dont il a la charge, sans coordination centrale.

Le modèle centralisé dispose d'une vue globale de toutes les ressources et de leurs dépendances, ce qui permet de mieux coordonner la réconciliation à travers plusieurs contrôleurs. Dans le cadre de ce

modèle, l'orchestrateur pourrait décider de ne pas réconcilier une ressource tant que ses dépendances ne sont pas prêtes. À l'inverse, dans le modèle décentralisé, la ressource serait quand même réconciliée quoi qu'il arrive il incomberait alors au contrôleur de décider si une dépendance non prête nécessite d'attendre ou non. Ce dernier modèle permet en revanche davantage de flexibilité.

Cette vue globale donne également au modèle centralisé un avantage quant à la réaction aux changements d'état et de spécification des ressources : un élément central coordonnant l'ensemble, il devient trivial de propager ces changements ou de redéclencher les réconciliations des ressources dépendantes. Un tel mécanisme de propagation, dans un modèle décentralisé, nécessiterait un mécanisme additionnel. La réaction aux événements externes (par exemple, un conteneur qui s'arrête, ou une interface réseau qui tombe) suit toutefois la logique inverse : elle est nativement plus simple à traiter dans le modèle décentralisé, tandis que le modèle centralisé nécessiterait l'ajout d'une procédure dédiée permettant de notifier l'orchestrateur.

Le modèle centralisé présente enfin deux avantages mineurs. D'une part, il permet d'assigner trivialement le parent d'une ressource : l'appel de réconciliation étant effectué sur une ressource connue, et sa réponse contenant les sous-ressources à créer, aucun risque de mauvaise assignation n'existe. D'autre part, il permet de détecter aisément la panne d'un contrôleur, celui-ci étant simplement considéré comme bloqué s'il ne répond pas dans le délai imparti.

Deux critères supplémentaires ont été écartés de la comparaison. Le premier concerne le nombre d'appels API : le modèle centralisé n'en requiert qu'un, contre au moins deux pour le modèle décentralisé (l'un pour récupérer la ressource, l'autre pour mettre à jour l'état) ce critère n'est pas retenu, car le nombre de ressources reste faible et les contrôleurs sont tous locaux. Le second concerne la robustesse : le modèle décentralisé limite la panne à un nombre restreint de ressources, mais ce critère n'est pas non plus retenu, les contrôleurs étant de toute façon primordiaux au fonctionnement du système, si bien que leur panne le rend inutilisable indépendamment du modèle choisi.

L'ensemble de ces points sont synthétisés dans le tableau 3 :

Critère	Pertinence	Faveur	Justification
Contrôle global de la planification	Moyenne	Centralisé	Une boucle unique peut optimiser la réconciliation en tenant compte des dépendances et de la hiérarchie des ressources.
Flexibilité de planification	Moyenne	Décentralisé	Chaque contrôleur peut régler son propre intervalle d'exécution, son backoff et sa concurrence sans coordination centrale.
Réaction aux événements internes	Moyenne	Centralisé	Déclencher une réconciliation sur changement d'état est trivial dans une boucle centrale; dans un modèle décentralisé, chaque contrôleur devrait gérer ses propres souscriptions.
Réaction aux événements externes	Moyenne	Décentralisé	Les événements externes (ex. arrêt d'un conteneur) peuvent être traités directement par le contrôleur concerné. Dans le modèle centralisé, ces signaux doivent transiter par l'orchestrateur.
Détection d'un contrôleur bloqué	Faible	Centralisé	L'orchestrateur central peut détecter un contrôleur bloqué via un timeout sur l'appel de réconciliation. Les contrôleurs décentralisés échouent silencieusement.
Assignation automatique du parent	Faible	Centralisé	La réponse de <code>reconcile()</code> contient les demandes de création, ce qui permet d'associer trivialement la relation parent-enfant.
Nombre d'appels API	Non retenu	Centralisé	<code>reconcile()</code> regroupe l'état en un seul appel, contre plusieurs requêtes par itération en mode décentralisé. Non pertinent étant donné le faible nombre de ressources et l'absence de contrainte de débit.
Périmètre d'une panne	Non retenu	Décentralisé	Une panne de l'orchestrateur central arrête toutes les réconciliations. En mode décentralisé, la panne est isolée au contrôleur concerné. Non pertinent car la panne de n'importe quel contrôleur met en péril le système entier.

Tableau 3. – Comparaison des modèles d'orchestration centralisés et décentralisés

Le modèle centralisé présente enfin une propriété plus fondamentale : la réconciliation d'une ressource repose alors uniquement sur des éléments entièrement déterminés par l'orchestrateur, à savoir sa spécification et son état, ce qui garantit un comportement reproductible d'un cycle à l'autre. Un contrôleur disposant de sa propre boucle pourrait, à l'inverse, introduire un état ou une logique de planification propre, non visible de l'extérieur, ce qui nuirait à cette prévisibilité.

Au regard de l'ensemble de ces éléments, le modèle centralisé a été retenu comme modèle d'orchestration. Sa propriété la plus importante réside dans la pureté de la réconciliation qu'il permet : celle-ci ne dépend alors que d'éléments entièrement déterminés par l'orchestrateur, à savoir la spécification et l'état de la ressource. Les avantages en termes de coordination des dépendances, de réaction aux événements internes et de détection de pannes renforcent encore ce choix, et l'emportent sur la flexibilité de planification et la capacité de réaction aux événements externes offerte par le modèle décentralisé. Les faiblesses du modèle centralisé (réaction aux événements externes spontanés, flexibilité de la planification) sont compensées par un mécanisme explicite de notification : chaque contrôleur peut signaler un événement externe à l'orchestrateur, qui place alors la ressource concernée en tête de la file d'attente. Les aspects de robustesse et de charge d'appels API sont considérés comme secondaires dans un contexte où les contrôleurs sont locaux et en nombre limité.

2.3.4. Boucle de réconciliation

L'orchestrateur maintient une file d'attente de ressources à réconcilier. Lors de l'insertion initiale d'une nouvelle ressource ou de la modification d'une ressource existante, celle-ci est placée dans la file si ses dépendances sont prêtes. Régulièrement, l'orchestrateur va prendre l'ensemble des ressources arrivées à échéance et les réconcilier.

La réconciliation demeure purement séquentielle : il n'y a jamais plus d'une ressource réconciliée en parallèle, ce qui exclut tout problème de concurrence lors de ce processus. Afin de ne pas bloquer la boucle indéfiniment, une durée maximale par réconciliation est imposée. En outre, lorsqu'une réconciliation change le statut d'une ressource, en particulier son passage à un statut prêt ou terminé, les ressources qui en dépendent sont à leur tour ajoutées à la file d'attente comme décrit dans le [chapitre 2.3.6, Réaction aux événements](#).

Afin d'éviter toute incohérence, l'ensemble des ressources sont verrouillées le temps d'une réconciliation. Toute modification de n'importe quelle ressource est suspendue jusqu'à ce que la réconciliation en cours soit terminée ou interrompue par expiration de la durée maximale impartie. Cela vaut aussi pour une mise à jour de la configuration du système par l'administrateur.

2.3.5. Suppression des ressources

Lorsqu'une ressource est supprimée, l'ensemble des ressources qu'elle possède est aussi supprimées. Afin de toujours garantir l'intégrité référentielle, la suppression part des feuilles. En effet, une ressource ne peut pas être complètement retirée du système tant qu'elle possède d'autres

ressources ou des dépendances entrantes. En outre, comme indiqué dans le [chapitre 2.2.4, Ressources mutualisées](#), les ressources mutualisées sont automatiquement placées en suppression dès lors qu'elles n'ont plus aucune dépendance entrante. La suppression de cette catégorie de ressource suit les mêmes règles que les autres ressources.

2.3.6. Réaction aux événements

Un événement, interne ou externe, se traduit par la (re)planification d'une ressource au sein de la file d'attente décrite dans le [chapitre 2.3.4, Boucle de réconciliation](#). Un événement interne survient lorsque l'administrateur transmet une nouvelle configuration : la ressource cible, une fois sa nouvelle spécification validée, est immédiatement replanifiée. Un événement interne peut aussi survenir lorsque le statut d'une ressource change après une réconciliation : les ressources qui en dépendent sont alors planifiées afin de réagir à ce changement de statut. Un événement externe survient lorsqu'un contrôleur constate, en dehors de tout appel de réconciliation, un changement d'état pertinent pour une ressource dont il a la charge, par exemple l'arrêt inattendu d'un conteneur il lui suffit alors de demander à l'orchestrateur de placer cette ressource dans la file. Dans les deux cas, le mécanisme reste identique du point de vue de l'orchestrateur, qui ne distingue pas l'origine de l'événement.

Chapitre 3

Implémentation

Ce chapitre présente les choix techniques retenus pour la réalisation du système, ainsi que les éléments d'implémentation significatifs. Les composants logiciels et les choix technologiques globaux au projet sont d'abord présentés, avant que l'implémentation de l'orchestrateur et des différents contrôleurs ne soit détaillée. L'API et ses clients d'administration sont ensuite décrits, suivis du système de fichiers racine, du processus de démarrage, ainsi que de l'installation et du chiffrement du système. Le chapitre se conclut par la description du système de build, de la configuration du noyau, de la génération de l'image finale du système, ainsi que de l'environnement de test et du pipeline d'intégration continue mise en place.

3.1. Composants

3.1.1. Choix des logiciels tiers

Limine [14] est retenu comme bootloader et Podman [15] comme runtime de conteneurs, ces deux choix ayant été établis lors du travail de semestre. Podman est retenu en raison de son bon fonctionnement en mode rootless ainsi que de sa légèreté, supérieure à celle des solutions alternatives considérées. Limine est de même retenu pour sa légèreté, tout en offrant un ensemble de fonctionnalités suffisamment complet pour les besoins du système.

3.1.2. Choix du langage

Rust est choisi comme langage principal pour l'implémentation du projet. Ce choix repose sur la nécessité de disposer d'un langage permettant d'effectuer aisément des appels système, contrainte partagée par le C, le C++ et le Go, langages également considérés. Le choix final se porte sur Rust en raison de la maîtrise préalable de ce langage, acquise avant le début du projet. Le langage Go est par ailleurs utilisé pour la partie du projet reposant sur Terraform, cet usage étant imposé par une contrainte propre à cet outil plutôt que par un choix indépendant. Les concepts fondamentaux de Rust nécessaires à la compréhension du reste de ce chapitre sont documentés dans « Rust by Example » [16], en particulier dans le chapitre consacré aux types standards [17].

3.1.3. Choix des bibliothèques

Le nombre de bibliothèques externes est volontairement restreint, dans le but de réduire la surface d'attaque du système et de conserver une maîtrise complète sur son comportement. Le projet, incluant tous les contrôleurs, ne compte ainsi que 30 dépendances directes, hors dépendances propres aux tests,

ce qui représente un total de 174 dépendances transitives⁵. La majorité de ces dépendances sont par ailleurs des dépendances standard dans un environnement Rust telles que le système de journalisation, la gestion d'erreur, l'asynchronicité, ou les interfaces pour les appels système.

La communication entre composants, détaillée dans le [chapitre 3.1.4, Communication entre les composants](#), repose sur gRPC [18]. Tonic [19] constitue, à ce jour, la seule implémentation gRPC pour Rust suffisamment mature et maintenue pour cet usage son adoption impose en retour le recours à Tokio [20] comme runtime asynchrone, ce dernier étant par ailleurs la bibliothèque d'exécution asynchrone la plus largement adoptée au sein de l'écosystème Rust, ce qui limite le risque lié à cette dépendance imposée. La sérialisation des messages échangés repose sur Serde [21], bibliothèque de facto standard au sein de l'écosystème Rust pour cet usage. Les bibliothèques restantes sont, pour leur part, propres à un contrôleur particulier et sont présentées avec celui-ci.

3.1.4. Communication entre les composants

Le protocole gRPC est utilisé uniquement comme mécanisme de transport et de définition des procédures exposées par chaque composant la structure des messages échangés n'est pas définie par ce protocole. Depuis sa ProtoBuf 3, celui-ci ne permet plus de marquer un champ comme obligatoire, dans le but de garantir la rétrocompatibilité entre un client et un serveur dont les versions diffèrent [22]. Cette contrainte n'est toutefois pas pertinente dans le cadre du présent système, l'ensemble des composants étant déployé simultanément et implémenté en Rust : le recours systématique à des champs optionnels y complexifierait inutilement le code. À la place, chaque message ne contient ainsi qu'un unique champ nommé `raw`, correspondant à la structure Rust concernée, sérialisée en JSON, plutôt que de reposer sur la structure de message générée par gRPC.

Deux services sont définis. Le service `StateService` permet à un contrôleur de communiquer vers l'orchestrateur, notamment afin de signaler la nécessité d'une réconciliation en dehors du cycle normal :

```
1 service StateService { protobuf
2     rpc ReconcileNow(ReconcileNowRequest) returns (ReconcileNowResponse) {}
3 }
```

Code 8. – Service gRPC pour l'orchestrateur

Source: <proto/containeros/state/v1/service.proto>

Ce service n'expose qu'une unique procédure, `ReconcileNow`, dont l'appel notifie l'orchestrateur qu'une ressource donnée doit être réconciliée immédiatement, sans attendre la prochaine échéance de la ressource. Cette procédure est notamment invoquée par un contrôleur lorsqu'un changement d'état

⁵Mesuré au moyen de la commande `cargo tree`.

externe à la spécification, tel qu'un événement survenant sur la ressource qu'il gère, rend le dernier état observé de cette ressource obsolète.

Le service `ReconcilerService`, implémenté cette fois par chaque contrôleur et appelé par l'orchestrateur, permet à ce dernier de déclencher la validation ou la réconciliation d'une ressource :

```
1 service ReconcilerService { protobuf
2   rpc Validate(ValidateRequest) returns (ValidateResponse) {}
3   rpc Reconcile(ReconcileRequest) returns (ReconcileResponse) {}
4 }
```

Code 9. – Service *gRPC* pour les contrôleurs

Source: [proto/containeros/reconciler/v1/service.proto](https://github.com/containeros/reconciler/v1/service.proto)

Ce service expose deux procédures distinctes, correspondant aux deux opérations décrites dans les chapitres [3.2.2, Réconciliation](#) et [3.2.3, Validation d'une ressource](#) . La procédure `Validate` est invoquée par l'orchestrateur préalablement à toute création ou mise à jour d'une spécification, afin d'obtenir du contrôleur concerné la confirmation de sa validité ainsi que, le cas échéant, sa spécification dérivée, tel qu'abordé dans le [chapitre 3.2.3, Validation d'une ressource](#). La procédure `Reconcile` est quant à elle invoquée afin de déclencher la réconciliation de la ressource par le contrôleur qui en a la charge.

3.1.5. Sécurité et isolation

Le travail de semestre préconisait l'isolation de chaque composant au sein de namespaces Linux distincts. Cette approche s'avère toutefois peu praticable dans le cadre de l'implémentation retenue : les contrôleurs doivent notamment partager un même namespace réseau ainsi qu'un même système de fichiers avec le reste du système. En l'absence d'une isolation complète par namespace, la réduction de la surface d'attaque de chaque composant repose sur la restriction de ses privilèges au moyen des capacités Linux [23]. L'ensemble des capacités est retiré par défaut de chaque composant seules celles strictement nécessaires à l'exécution d'un contrôleur donné, telles que `CAP_NET_ADMIN` pour le contrôleur réseau, lui sont accordées. Ces capacités ne sont en outre activées que durant l'exécution d'une réconciliation, et non de manière permanente durant l'ensemble du cycle de vie du composant, ce qui réduit la durée durant laquelle ces permissions sont exposées.

3.2. Orchestrateur

La mise à jour groupée d'un ensemble de ressources, invoquée par l'API lors de la réception d'une nouvelle configuration, est prise en charge par la procédure `bulk_upsert`. Cette procédure calcule d'abord, par comparaison entre l'ensemble des ressources précédemment connues et l'ensemble des ressources nouvellement soumises, trois catégories : les ressources ajoutées, absentes de l'état précédent les ressources supprimées, absentes de la nouvelle configuration et les ressources

modifiées, présentes dans les deux ensembles, mais dont la spécification diffère. Les ressources ajoutées et modifiées sont ensuite validées, tel que décrit dans le [chapitre 3.2.3, Validation d'une ressource](#) puis enregistrées.

La procédure `schedule_available` détermine enfin, parmi les ressources ainsi ajoutées ou modifiées, celles dont l'ensemble des dépendances est satisfait : une dépendance est considérée satisfaite si la ressource correspondante est elle-même dans un état prêt ou terminé. Seules les ressources dont l'ensemble des dépendances est ainsi satisfait sont effectivement planifiées pour réconciliation immédiate au sein de la file d'attente, les autres demeurant en attente jusqu'à ce que leurs dépendances respectives atteignent un état compatible.

3.2.1. File d'attente

Comme indiqué dans le [chapitre 2.3.4, Boucle de réconciliation](#), la boucle de réconciliation dépend d'une file d'attente, structure représentée ici par `Queue<K>`. Cette file est implémentée de manière générique : elle permet de planifier n'importe quel élément de type `K` à un moment précis. Dans le cadre du présent système, `K` représente l'identifiant unique d'une ressource. Son état interne est principalement inclus dans la structure `QueueInner<K>` illustrée dans le code 10 :

```
1 struct QueueInner<K> {  
2     scheduled: HashMap<K, Instant>,  
3     queue: BTreeMap<Instant, HashSet<K>>,  
4 }
```

 Rust

Code 10. – Structure interne de la file d'attente

Source: rust/cmd/state-manager/src/queue.rs

Les ressources ainsi planifiées sont stockées dans un dictionnaire basé sur un B-arbre (`BTreeMap`) [24], indexé par l'instant auquel leur réconciliation est prévue. Le recours à un B-arbre facilite la récupération des ressources dont la planification est arrivée à échéance : la date de réconciliation étant une valeur numérique ordonnée, toutes les valeurs inférieures à l'instant présent correspondent à des échéances passées et peuvent être traitées. L'implémentation Rust des B-arbres permet en outre de scinder l'arbre en deux à partir d'une clé donnée, ce qui correspond précisément au cas d'utilisation recherché. L'arbre étant par ailleurs ordonné, cela permet de parcourir les éléments en commençant par le plus ancien, sans recourir à une étape de tri additionnelle.

La valeur associée à une clé n'est pas un identifiant unique, mais un ensemble d'identifiants. En effet, lorsqu'une nouvelle configuration est soumise, l'ensemble des ressources nouvellement définies doit être planifié simultanément pour réconciliation c'est pourquoi chaque clé temporelle est associée à un ensemble d'identifiants (`HashSet`) [25] plutôt qu'à un identifiant unique. Un champ supplémentaire, `scheduled`, complète cette structure en fournissant un index inversé permettant de déterminer rapidement si, et à quel moment, une ressource donnée est planifiée.

Cette file expose la méthode asynchrone `drain_expired()`, qui attend qu’une ou plusieurs clés arrivent à échéance avant de les retourner. L’attente est passive : la fonction calcule le délai jusqu’à la prochaine échéance connue, puis se met en pause jusqu’à ce délai. Pour gérer le cas où un élément serait planifié pendant cette attente, avec une échéance plus proche que celle déjà calculée, ou alors qu’aucune échéance n’existait, un canal de notification est utilisé, basé sur la structure `Notify` du runtime asynchrone Tokio.

Étant donné que plusieurs opérations d’écriture (ajout individuel, ajout en masse, replanification) peuvent modifier l’échéance la plus proche et donc nécessiter une notification, ces opérations passent par un `QueueInnerGuard` illustré dans le code 11 :

```
1 struct QueueInnerGuard<'a, K> {
2     guard: RwLockWriteGuard<'a, QueueInner<K>>,
3     notify: Arc<Notify>,
4     earliest: Option<Instant>,
5 }
```

 Rust

Code 11. – Encapsulation de la file d’attente pour les opérations d’écriture

Source: rust/cmd/state-manager/src/queue.rs

Au moment de sa construction, ce guard capture l’échéance la plus proche connue avant modification. Lorsqu’il est détruit (`Drop` [26], l’équivalent Rust d’un `defer` en Go ou d’un destructeur en programmation orientée objet), il compare cette échéance initiale à l’échéance courante et déclenche une notification si celle-ci a changé, que ce soit parce qu’une échéance plus proche a été introduite, ou parce que la file est passée d’un état non vide à vide ou inversement. Cette approche permet de simplifier la gestion de l’envoi des notifications. En rendant ce processus entièrement transparent du point de vue du code modifiant la file d’attente.

Enfin, la file est encapsulée dans une structure `Queue`, qui fournit un accès concurrent aux différentes méthodes, illustré dans le code 12 :

```
1 struct Queue<K> {
2     queue: Mutex<QueueInner<K>>,
3     notify: Arc<Notify>,
4 }
```

 Rust

Code 12. – Encapsulation de la file d’attente pour un accès concurrent

Source: rust/cmd/state-manager/src/queue.rs

L’accès à `QueueInner` est protégé par un `Mutex` [27], chaque opération exposée par `Queue` nécessitant de toute façon un accès exclusif à la structure sous-jacente.

3.2.2. Réconciliation

La réconciliation est une tâche de fond qui récupère, en boucle, l'ensemble des identifiants arrivés à échéance via `Queue::drain_expired`, puis transmet une requête au contrôleur responsable afin de réconcilier chaque ressource. Cette requête reprend la structure de ressource complète, décrite dans le code 13 :

```

1  pub struct Resource<Spec, DerivedSpec, State> {
2      pub id: Identity,
3      pub phase: Phase,
4      pub status: Status,
5      pub spec: Spec,
6      pub derived_spec: DerivedSpec,
7      pub state: Option<State>,
8      pub children: Vec<TerminalResource<Value, Value, Value>>,
9      pub dependencies: Vec<TerminalResource<Value, Value, Value>>,
10     pub dependents: Vec<TerminalResource<Value, Value, Value>>,
11 }

```

Code 13. – Structure d'une ressource transmise lors de la réconciliation

Source: rust/crates/cos-proto-reconciler/src/lib.rs

Dans le code 13, les champs `children`, `dependencies` et `dependents` reposent sur le type `TerminalResource`, une variante de `Resource` dans laquelle ces mêmes champs sont réduits à de simples identifiants, ce qui évite la récursion de la structure, comme illustré dans le code 14 :

```

1  pub struct TerminalResource<Spec, DerivedSpec, State> {
2      pub id: Identity,
3      pub phase: Phase,
4      pub status: Status,
5      pub spec: Spec,
6      pub derived_spec: DerivedSpec,
7      pub state: Option<State>,
8      pub children: HashSet<Identity>,
9      pub dependencies: HashSet<Identity>,
10     pub dependents: HashSet<Identity>,
11 }

```

Code 14. – Structure d'une ressource terminale

Source: rust/crates/cos-proto-reconciler/src/lib.rs

Le message de réconciliation contient une ressource sous forme générique, le type `Value` représentant un type générique susceptible d'être converti ultérieurement en un type connu, ce mécanisme permettant à une ressource de posséder des ressources enfants de types différents. Le contrôleur

déséréalise d’abord cette ressource générique afin d’en identifier le schéma, puis déséréalise les champs génériques dans la structure finale correspondante. La réconciliation d’une ressource est déclenchée toutes les 30 secondes, ou plus tôt si le contrôleur notifie l’orchestrateur d’une nécessité de réconciliation anticipée.

Les ressources arrivées à échéance sont réconciliées séquentiellement, dans l’ordre de leur ancienneté, sans regroupement. Deux ressources consécutives destinées au même contrôleur font ainsi l’objet de deux requêtes distinctes, la seconde n’étant transmise qu’après réception de la réponse à la première. Un échec affectant la réconciliation d’une ressource n’empêche pas la tentative de réconciliation de la ressource suivante.

La réponse du contrôleur inclut le nouvel état, le statut, l’ensemble des enfants, et l’ensemble des dépendances, dont la structure est décrite dans le code 15 :

```
1 pub struct ResourceResponse<State> {  
2     pub status: Status,  
3     pub state: Option<State>,  
4     pub children: Vec<SubResource<Value>>,  
5     pub dependencies: HashSet<Identity>,  
6 }
```

Code 15. – Structure de réponse d’une réconciliation

Source: rust/crates/cos-proto-reconciler/src/lib.rs

Le champ `status` indique le résultat de la réconciliation, tel que prêt, pas prêt ou une erreur. Le champ `state`, optionnel, correspond au nouvel état de la ressource lorsque la réconciliation en produit un son absence signifie que l’état de la ressource reste inchangé. Le champ `children` contient l’ensemble des ressources enfants créées ou maintenues par le contrôleur à l’issue de cette réconciliation. Le champ `dependencies` contient quant à lui l’ensemble des ressources dont dépend la ressource réconciliée. Seul l’identifiant est transmis pour les dépendances, tandis que l’identifiant et la spécification sont transmis pour les enfants, ces derniers pouvant être nouvellement créés et donc inconnus de l’orchestrateur.

Deux catégories d’erreur sont distinguées lors de la réconciliation : l’erreur de protocole et l’erreur de logique. Une erreur de protocole survient lorsque les données échangées via gRPC sont invalides ou non interprétables, ou lorsqu’une erreur imprévue se produit au niveau du transport. Une erreur de logique, en revanche, correspond à un échec géré par le contrôleur dans le cadre normal de la réconciliation, et se traduit par un statut d’erreur dans le champ `status` de la réponse. Dans ce dernier cas, la réconciliation est considérée comme réussie du point de vue du protocole gRPC, indépendamment du contenu du statut. En cas d’erreur de protocole, l’orchestrateur attribue lui-même

à la ressource un statut d'erreur qualifié de « transport » dans tous les autres cas, le statut attribué à la ressource correspond à celui fourni par le contrôleur dans sa réponse.

3.2.3. Validation d'une ressource

La validation d'une ressource s'effectue à chaque création ou mise à jour d'une spécification. Lorsque plusieurs ressources sont ajoutées ou modifiées simultanément, l'ensemble de ces ressources est validé en parallèle. Une erreur affectant un seul élément de cet ensemble entraîne l'échec de la validation pour la totalité des ressources concernées.

La validation d'une ressource s'effectue à travers la procédure `validate()`. La requête contient d'abord la nouvelle spécification de la ressource, et si cette ressource existe déjà, la spécification courante de celle-ci ainsi que son état. La ressource actuelle est transmise, car certaines ressources, telles que la ressource permettant d'installer le système sur le disque, sont totalement ou partiellement immuables leur modification doit alors passer par la recréation d'une nouvelle ressource.

La ressource préexistante est optionnelle dans la requête de validation, son absence correspondant au cas d'une première réconciliation, pour laquelle aucun état antérieur n'existe encore. Par ailleurs, certains champs de la spécification peuvent être dérivés intégralement à partir de la spécification elle-même, sans intervention de l'utilisateur. Le contrôleur fournit alors, en réponse à la validation, une spécification dite dérivée, calculée uniquement à partir de la spécification soumise. Cette spécification dérivée n'est recalculée que lorsque la spécification change elle ne peut pas être modifiée lors d'une réconciliation normale. Ce mécanisme dispense l'utilisateur de renseigner manuellement des champs entièrement déductibles, tout en évitant un recalcul systématique de ces champs à chaque réconciliation. La garantie que la spécification dérivée existe à chaque réconciliation permet en outre au contrôleur de traiter ce champ comme systématiquement disponible, ce qui allège la logique de réconciliation en dispensant celle-ci de toute vérification de présence des champs dérivés.

3.3. Contrôleurs

Chaque contrôleur implémente le service gRPC `ReconcilerService` décrit précédemment, en particulier les procédures `validate` et `reconcile`, selon une approche commune. La réception d'une requête entraîne d'abord la désérialisation de la ressource générique reçue, puis un branchement est effectué sur le schéma de cette ressource, comme illustré dans le code 16 pour la procédure `reconcile` du contrôleur réseau :

```

1  async fn reconcile(
2      &self,
3      request: Request<ReconcileRequest>,
4  ) -> Result<Response<ReconcileResponse>, Status> {
5      let req = request.into_inner();
6      let resource: Resource<Value, Value, Value> =
7          serde_json::from_slice(&req.raw)
8              .map_err(|err| Status::from_error(err.into()))?;
9
10     match resource.id.key().schema.as_ref() {
11         "network:link" => {
12             reconcile!(resource, LinkResource, {
13                 let (conn, handle, _) = new_connection()?;
14                 tokio::spawn(conn);
15                 LinkReconciler::new_with(handle)
16             });
17         }
18         // ...
19         _ => return Err(Status::not_found("schema does not exist")),
20     }
21 }

```

Code 16. – Aiguillage sur le schéma d’une ressource

Source: [rust/cmd/network-controller/src/main.rs](https://github.com/rust/cmd/network-controller/src/main.rs)

Chaque branche déserialise la ressource générique dans la structure typée correspondant au schéma identifié, instancie le sous-contrôleur (Reconciler) responsable de ce type de ressource avec les paramètres adéquats, puis lui délègue le traitement de la requête. La macro `reconcile!` permet de masquer une grande partie de la logique liée à la désérialisation de la ressource.

La logique de réconciliation propre à chaque type de ressource suit un déroulement identique, structuré en quatre étapes : la spécification de la ressource est d’abord validée l’état physique courant de la ressource est ensuite récupéré (`refresh`) un plan d’action est calculé par comparaison entre cet état et la spécification (`plan`) ce plan est enfin exécuté (`apply`), avant qu’un second `refresh` ne détermine l’état résultant réel de la ressource. Chacune de ces étapes est susceptible d’échouer indépendamment des autres un échec à n’importe quelle étape interrompt immédiatement la réconciliation et retourne une réponse dont le champ `status` est mis à `Error`, tout en conservant, autant que possible, le dernier état connu de la ressource.

Le statut final d’une réconciliation ayant abouti dépend du plan calculé lors de la seconde évaluation et de l’état courant de la ressource : l’absence de toute action à entreprendre (`NoOp`) alors que la ressource est en cours de suppression indique qu’elle a été supprimée avec succès ou qu’elle

n’a jamais existé en dehors de tout contexte de suppression, l’absence d’action correspond au statut prêt. À l’inverse, la persistance d’un plan non vide à l’issue de son exécution indique que, malgré les actions correctives entreprises, l’état réel de la ressource ne correspond toujours pas à la spécification attendue, et aboutit à un statut pas prêt. Certaines ressources peuvent également retourner un statut terminé plutôt que prêt, lorsque des réconciliations ultérieures n’auraient pas de sens, par exemple un conteneur dont la politique de redémarrage est « never ».

3.3.1. Contrôleur système

Le contrôleur système, implémenté dans [rust/cmd/system-controller](#), prend actuellement en charge la seule gestion du contenu du répertoire `/etc/`⁶, à travers la ressource `system:etc`⁷.

L’écriture d’un fichier au sein de `/etc/` est effectuée de manière atomique. Lorsque la spécification de la ressource change, un nouveau fichier temporaire est créé au moyen du flag `O_TMPFILE` [28] : ce mécanisme permet de créer un descripteur de fichier associé à un inode dépourvu de tout lien dans l’arborescence, garantissant qu’aucun processus tiers ne puisse observer ou accéder à ce fichier avant que son contenu ne soit intégralement écrit. L’ensemble des opérations nécessaires (écriture du contenu, application des permissions, etc.) est ensuite effectué sur ce fichier temporaire. Une fois ces opérations terminées, le fichier est rendu visible dans l’arborescence au moyen de `linkat` [29], qui associe l’inode déjà constitué à son chemin final : cette opération remplace atomiquement le fichier existant, le cas échéant, sans jamais exposer d’état intermédiaire.

Le chemin spécifié dans la spécification de la ressource doit désigner un fichier réel, sans aucun lien symbolique, et ne comporter aucun élément relatif permettant de sortir du répertoire attendu. Cette contrainte est imposée par l’appel `openat2` [30], combiné aux flags de résolution `RESOLVE_NO_SYMLINKS` et `RESOLVE_BENEATH`, qui interdisent respectivement la traversée de tout lien symbolique rencontré lors de la résolution du chemin, et toute sortie du répertoire racine désigné par ce même appel. Cette double contrainte prévient toute exploitation, par une spécification malveillante ou erronée, d’un lien symbolique ou d’une séquence relative telle que « `..` » afin d’accéder à un fichier situé en dehors du répertoire visé. Cet appel retournant un descripteur de fichier, celui-ci est ensuite réutilisé pour créer le fichier temporaire dans ce même répertoire, sans nouvelle résolution de chemin, ce qui évite toute race condition entre la validation et la création. Cela permet d’appliquer le principe « Time of Check, Time of Use » (TOCTOU) [31].

3.3.2. Contrôleur réseau

Le contrôleur réseau, implémenté dans [rust/cmd/network-controller](#), gère l’ensemble des ressources du domaine réseau. L’ensemble de ces ressources communique avec le noyau au moyen du protocole `netlink` [32], via la bibliothèque Rust `rtnetlink` [33]. Les ressources `network:dns` et `network:address`

⁶Par exemple, les certificats des autorités de certification racine, tels que Cloudflare, SwissSign, etc.

⁷Implémentée dans [rust/cmd/system-controller/src/resources/etc-file.rs](#).

ne présentent aucune particularité notable : l'état physique correspondant est d'abord récupéré, puis comparé à la spécification, avant d'être créé s'il est absent.

La ressource `link`⁸ se distingue par le nombre élevé d'attributs qui la composent. De plus, l'API exposée par la `netlink` ne retourne pas directement une structure représentant un lien réseau, mais un message contenant une liste d'attributs hétérogènes, dont seul un sous-ensemble est pertinent, et dont le sous-ensemble effectivement présent varie selon le lien concerné, comme illustré dans le code 17 :

```
1 vec![
2     LinkAttribute::Name(/* ... */),
3     LinkAttribute::OperState(/* ... */),
4     LinkAttribute::Index(/* ... */),
5     // many more...
6 ]
```

Code 17. – Extrait d'une liste d'attributs retournée pour un lien réseau

Source: [rust/cmd/network-controller/src/resources/link.rs](https://github.com/rust-netlink/rust-netlink/blob/master/src/resources/link.rs)

Or, en Rust, l'instanciation d'une structure requiert de fournir une valeur pour chacun de ses champs, ce qui empêche de construire directement la structure finale, illustrée dans le code 18, à partir d'une liste d'attributs incomplète :

```
1 #[derive(Builder /*, ... */)]
2 #[builder(pattern = "mutable", vis = "pub(crate)")]
3 pub struct LinkState {
4     pub index: u32,
5     pub running: bool,
6     pub admin_up: bool,
7     pub oper_state: LinkOperState,
8     // ...
9 }
```

Code 18. – Structure finale représentant l'état d'un lien réseau

Source: [rust/cmd/network-controller/src/resources/link.rs](https://github.com/rust-netlink/rust-netlink/blob/master/src/resources/link.rs)

Rendre l'ensemble des champs de cette structure optionnels ne constitue pas une solution satisfaisante, la plupart de ces champs étant en réalité obligatoires dans l'état final d'un lien réseau. Un pattern « builder » est employé pour résoudre ce problème : la macro `Builder`, appliquée à `LinkState`, génère automatiquement une structure intermédiaire, `LinkStateBuilder`, illustrée dans le code 19, dans laquelle chaque champ est rendu optionnel :

⁸Implémentée dans [rust/cmd/network-controller/src/resources/link.rs](https://github.com/rust-netlink/rust-netlink/blob/master/src/resources/link.rs).

```

1 pub struct LinkStateBuilder {
2     pub index: Option<u32>,
3     pub running: Option<bool>,
4     pub admin_up: Option<bool>,
5     pub oper_state: Option<LinkOperState>,
6     // ...
7 }

```

Code 19. – Structure intermédiaire générée par la macro *Builder*

Source: rust/cmd/network-controller/src/resources/link.rs

Cette structure intermédiaire évite au contrôleur de devoir gérer manuellement, au moyen d’une boucle et de variables intermédiaires dédiées, l’accumulation progressive des attributs reçus avant de pouvoir construire la structure finale. Chaque attribut reconnu dans le message netlink renseigne directement le champ correspondant de cette structure intermédiaire, comme illustré dans le code 20 :

```

1 fn build_from_attrs(
2     &mut self, // self: LinkStateBuilder
3     attrs: &[LinkAttribute]
4 ) -> Result<()> {
5     // ...
6
7
8     for attr in attrs {
9         match attr {
10             LinkAttribute::Mtu(mtu) => { self.mtu(*mtu); }
11             LinkAttribute::OperState(s) =>
12                 { self.oper_state((*s).into()); }
13             // ...
14             _ => {}
15         }
16     }
17
18     // ...
19 }

```

Code 20. – Extrait du constructeur de la ressource *link*

Source: rust/cmd/network-controller/src/resources/link.rs

Une fois l’ensemble des attributs du message traités, la structure intermédiaire *LinkStateBuilder* est converti vers la structure finale *LinkState*, cette conversion échouant si l’un des champs obligatoires de *LinkState* demeure non spécifié à l’issue du traitement.

La ressource `network:route` présente une particularité : avec la librairie `rtnetlink`, la récupération d'une route unique par son identifiant ne retourne pas l'entrée correspondante, mais tente de résoudre la route associée à cet identifiant, ce qui ne correspond pas au comportement recherché. Le contrôleur récupère par conséquent l'ensemble des routes existantes, puis filtre ce résultat afin d'isoler celle qui correspond à la ressource gérée.

La ressource `network:dhcp` repose sur la bibliothèque `smoltcp` [34], seule bibliothèque Rust fournissant un client DHCP réutilisable comme composant logiciel indépendant. Le protocole DHCP étant par nature asynchrone et piloté par des échéances propres au serveur DHCP plutôt que par le cycle de réconciliation du contrôleur, la première réconciliation de cette ressource se limite à démarrer une tâche d'arrière-plan chargée de piloter le client DHCP. Lorsque cette tâche reçoit une nouvelle configuration du serveur, elle l'enregistre, puis notifie l'orchestrateur. L'orchestrateur replanifie alors la réconciliation de la ressource, à l'occasion de laquelle le contrôleur récupère la configuration ainsi obtenue et crée les sous-ressources `network:address` et `network:route` correspondantes.

3.3.3. Contrôleur de conteneurs

Le contrôleur de conteneurs, implémenté dans [rust/cmd/container-controller](#), s'appuie sur Podman comme runtime de conteneurs, et sur la bibliothèque Bollard [35] pour communiquer avec celui-ci. Lors de la création d'un runtime, un port lui est attribué de manière arbitraire, sur lequel Podman expose son API, utilisée ensuite par le contrôleur pour l'ensemble des opérations relatives aux ressources du domaine de la conteneurisation.

3.4. API et clients d'administration

L'API constitue l'unique surface d'administration du système et se repose aussi sur gRPC. Les procédures disponibles correspondent à celles présentées dans le [tableau 2, Liste commandes disponible](#) présenté dans le chapitre 1. Chaque procédure exposée par le service `ApiService` est précédée d'une vérification d'authentification (`auth_or_fail`), qui interrompt la requête avant tout traitement en cas d'échec.

La procédure `push_config` constitue le point d'entrée principal de l'API : elle reçoit l'ensemble des ressources composant la configuration déclarative du système tel que présenté dans le [chapitre 1, Présentation du système](#). Cette procédure identifie, parmi ces ressources, celle de schéma `api`, dont la présence est obligatoire, ainsi qu'une éventuelle ressource de schéma `install`. La configuration de l'API elle-même est mise à jour, puis l'ensemble des ressources soumises remplace, par une opération de mise à jour groupée (`bulk_upsert`), les ressources précédemment connues du système les ressources absentes de la nouvelle configuration sont ainsi implicitement supprimées. Lorsque le système se trouve en mode maintenance et qu'une ressource `install` est présente, cette procédure déclenche en outre, une fois la mise à jour des ressources effectuée, l'installation du système sur le

disque, suivie d'un redémarrage automatique après un court délai, processus décrit plus en détail dans le [chapitre 3.7, *Installation et chiffrement*](#).

3.5. Système de fichier racine

Le système repose sur un fonctionnement presque entièrement immuable. Le système de fichiers racine contient le strict minimum, à savoir les binaires dans `/bin/` et quelques fichiers statiques dans `/etc/`. Ces deux répertoires sont fournis au système à travers une image d'un système de fichiers de type SquashFS [36] qui les rend totalement immuables. Cette image représente le système de fichiers racine (`/`). Toutefois, certains fichiers doivent pouvoir être écrits dans `/etc/` durant le fonctionnement normal du système, par exemple pour configurer la résolution DNS ou Podman. Afin de permettre cela, un système de fichiers temporaire de type Tmpfs [37] est superposé à l'image SquashFS au moyen d'OverlayFS [38]. Ce système de fichiers temporaire est entièrement persisté en mémoire ainsi, lorsque le système d'exploitation redémarre, l'ensemble de son contenu est perdu. Ceci ne constitue pas un problème compte tenu du modèle déclaratif du système : l'ensemble de ces fichiers est en réalité dérivé de la configuration, et recréé identique à chaque redémarrage.

3.6. Démarrage du système

Le bootloader, quel qu'il soit, charge le noyau ainsi que l'`initrd` en mémoire. L'`initrd` est un système de fichiers racine, entièrement chargé en mémoire. En raison des contraintes mémoires et de la taille du système de fichiers racine réel, l'`initrd` du présent système est un système de fichiers minimal, dont le seul rôle est de préparer le système de fichiers racine réel décrit au [chapitre 3.5, *Système de fichier racine*](#). Une fois le noyau démarré, celui-ci exécute le programme `/init` se trouvant sur l'`initrd` ce programme devient alors le processus initial (PID 1) du système. Dans le cas du présent système, ce programme est le seul présent sur ce système de fichiers. Ce programme, implémenté dans [rust/cmd/init](#), a pour but de localiser l'image SquashFS du système de fichiers racine réel, où qu'elle se trouve, de la préparer, puis d'y apposer la surcouche d'écriture au moyen d'OverlayFS.

Pour localiser le système de fichiers racine réel, l'`/init` se base sur le paramètre de démarrage `cos.bootdisk`, qui spécifie le disque et le numéro de partition sur lesquels se trouvent les artefacts de démarrage. L'`/init` monte alors temporairement cette partition afin d'y récupérer l'image SquashFS. Si ce paramètre est absent, l'`/init` considère qu'il est démarré depuis l'image ISO, et tente de monter cette dernière afin d'y récupérer l'image. Dans les deux cas, l'image se nomme `root.squashfs` et se trouve à la racine respective de son support.

Une fois le système de fichiers racine réel monté, l'`/init` passe la main au superviseur, implémenté dans [rust/cmd/supervisor](#), responsable de monter le reste du système de fichiers. Cette répartition des rôles entre `/init` et le superviseur s'explique par le fait que l'`initrd`, contenant l'`/init`,

est chargé en mémoire et doit à ce titre demeurer aussi léger que possible. Le superviseur suit ensuite les étapes illustrées dans la figure 9 :

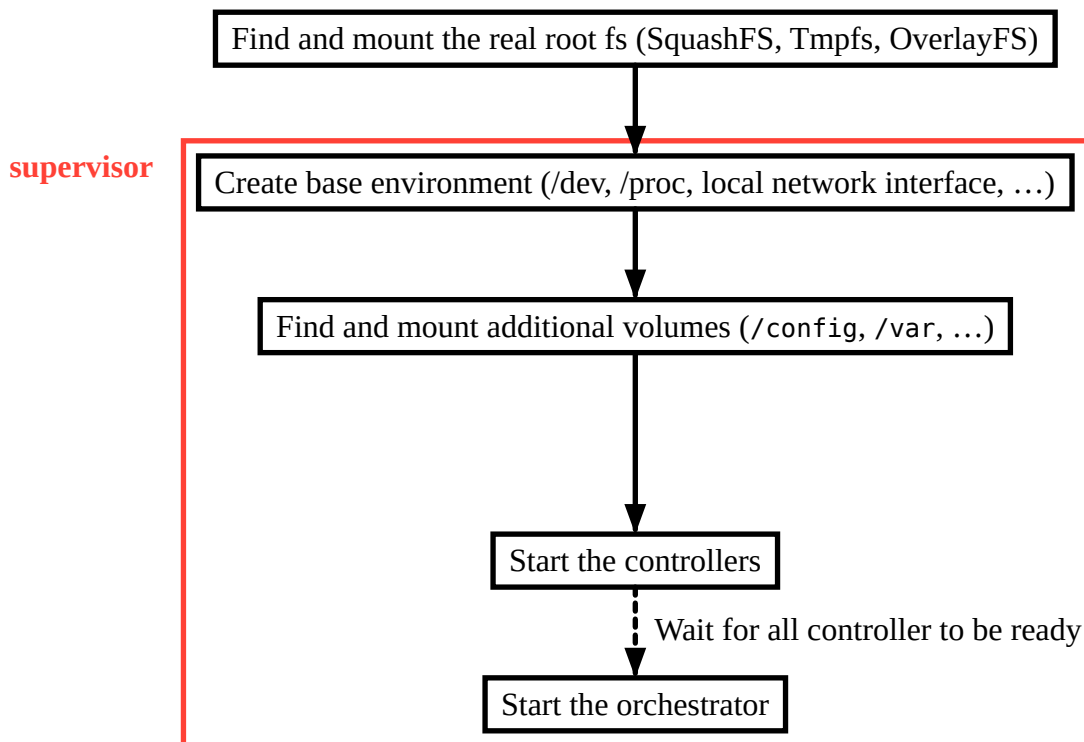


Figure 9. – Démarrage du système

Illustre les étapes principales du démarrage du système, avant que celui-ci n'entame la réconciliation.

Le superviseur construit l'arborescence de fichiers standard de Linux telle que spécifiée par le Filesystem Hierarchy Standard [39]. Il met ensuite en place le réseau local de l'hôte, l'interface `localhost` n'étant pas activée par défaut sous Linux cette interface est essentielle au fonctionnement du système, les différents contrôleurs communiquant entre eux via une adresse locale. Puis monte les différents volumes additionnels spécifiés dans les paramètres de démarrage, tels que `/config/` via `cos.configdisk` ou `/var/` via `cos.datadisk`. Le superviseur démarre alors ces contrôleurs et attend qu'ils soient prêts avant de démarrer l'orchestrateur. À partir de ce point, le superviseur n'a plus d'autre rôle que d'attendre l'extinction du système.

Si l'une de ces étapes échoue, pour quelque raison que ce soit, le programme s'interrompt et déclenche un « kernel panic ». À ce stade du cycle de vie du système, les composants permettant une gestion déclarative ne sont pas encore chargés et il n'y a donc aucune autre possibilité pour joindre le système.

La figure 10 illustre un tel comportement, provoqué par l'indisponibilité du disque de configuration attendu par le système :

```

2026-08-05T15:12:46.434646Z TRACE linux_utils: checking /sys/fs/cgroup
2026-08-05T15:12:46.437279Z TRACE linux_utils: mounting /sys/fs/cgroup as cgroup2
2026-08-05T15:12:46.444171Z INFO supervisor::linux_init: mounting /dev/vda3
[ 1.135197] /dev/vda3: Can't lookup blockdev
Error: mounting /dev/vda3

Caused by:
  No such file or directory (os error 2)
[ 1.139069] Kernel panic - not syncing: Attempted to kill init! exitcode=0x00000100
[ 1.141071] CPU: 0 UID: 0 PID: 1 Comm: supervisor Not tainted 6.19.9 #1-NixOS PREEMPT(voluntary)
[ 1.143830] Hardware name: QEMU Standard PC (i440FX + PIIX, 1996), BIOS 1.16.2-debian-1.16.2-1 04/01/2014
[ 1.145681] Call Trace:
[ 1.146194] <TASK>

```

Figure 10. – Panique du noyau lorsqu'un disque n'est pas disponible

Le noyau tente de monter le disque de configuration, mais celui-ci n'est pas disponible alors qu'il devrait l'être.

En l'absence du disque de configuration, le système ne dispose d'aucune spécification à réconcilier et son exécution ne présenterait aucune utilité, tout en risquant d'exposer le système à des risques inutiles. La figure 11 illustre l'arbre des processus démarrés par le système, ainsi que les relations de filiation entre ceux-ci :

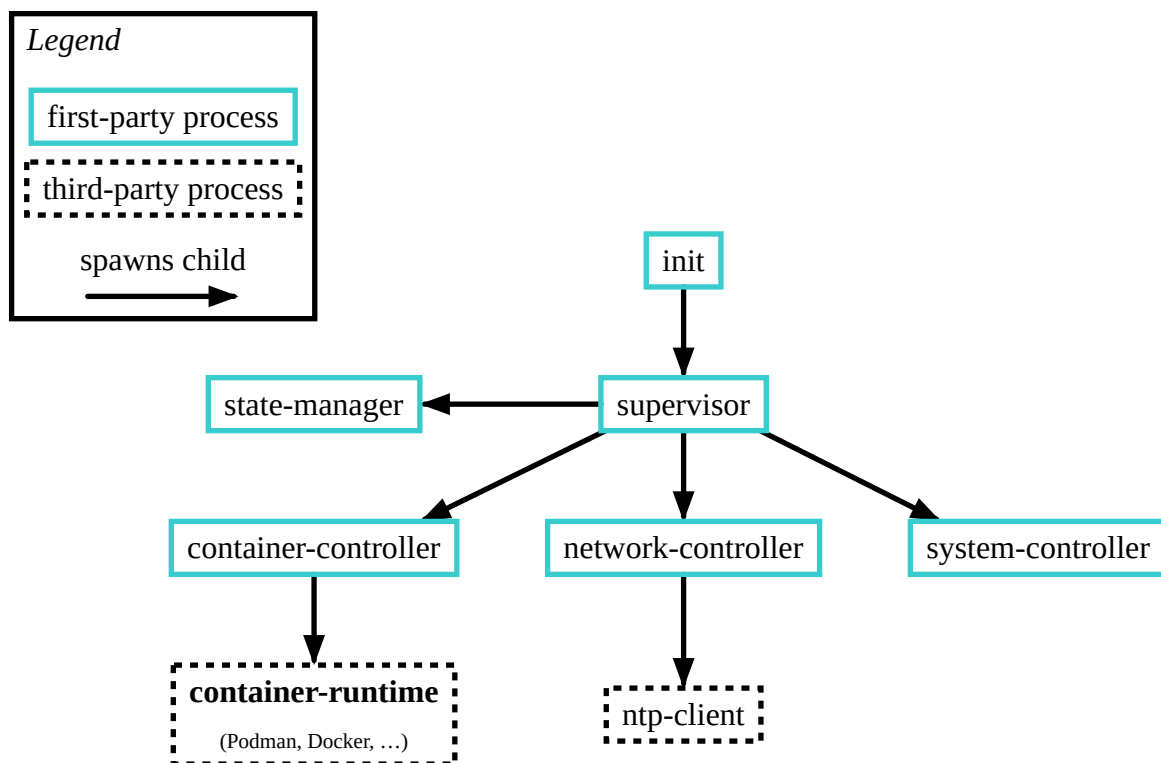


Figure 11. – Arbre d'exécution des processus du système

Illustre l'ordre dans lequel les processus du système sont démarrés, et qui les lance.

Le processus `/init` démarre le superviseur, responsable de lancer les autres processus, en commençant par les contrôleurs puis l'orchestrateur. Les différents contrôleurs peuvent, à leur tour, démarrer divers processus, tels qu'un runtime de conteneur ou un client DHCP.

3.7. Installation et chiffrement

L'installation du système sur disque est déclenchée par l'API, via la ressource `install`, et prise en charge par une procédure d'installation qui ne s'exécute que si le système est démarré en mode maintenance.

Lorsqu'une configuration comportant une ressource `install` est soumise à l'API, l'ensemble des ressources qu'elle contient est d'abord validé, puis enregistré dans l'état interne du système, tel que décrit au [chapitre 3.4, API et clients d'administration](#). La file d'attente de réconciliations est toutefois suspendue avant que cette installation ne soit commencée, ce qui empêche toute réconciliation d'une ressource et garantit que l'installation s'exécute sur un état stable.

Le disque système est partitionné selon un schéma composé de quatre partitions : une partition de boot de 1 MiB, une partition contenant les fichiers de Limine ainsi que l'image SquashFS du système, le noyau et l'initrd (limine, 512 MiB), une partition optionnelle destinée au disque de configuration (config, 10 MiB), et une partition optionnelle de données occupant l'espace restant (data). Limine est ensuite installé sur le disque, les partitions requises sont formatées et montées, avant que les fichiers de démarrage extraits de l'image ISO ne soient copiés vers la partition limine. Une configuration de démarrage est enfin générée, définissant deux entrées : l'une destinée à l'exécution normale du système, référençant les disques présents par leur chemin respectif au moyen de paramètres de démarrage dédiés, l'autre destinée au mode maintenance.

Le chiffrement des disques repose sur LUKS [40], combiné à Clevis [41] pour l'automatisation du déchiffrement. Le mode de chiffrement appliqué à chaque disque est spécifié indépendamment, via un paramètre de démarrage dédié de la forme `cos.<disktype>.encryption`, où `<disktype>` désigne l'un des disques gérés par le système, tel que `configdisk` ou `datadisk`.

3.8. Système de build

L'environnement de build repose sur l'outil Nix. Une distinction est requise entre trois usages du terme « Nix » : Nix en tant que système de build, Nix en tant que gestionnaire de paquets, et Nix en tant que distribution Linux (NixOS) [7]. Seul le premier usage, complété partiellement par le second, est utilisé dans le cadre de ce projet. Les concepts essentiels de Nix nécessaires à la compréhension de ce chapitre sont présentés à l'annexe C.

Le recours à Nix vise à fournir un environnement stable entre la machine de développement locale et l'environnement d'intégration continue. Nix permet non seulement la construction des artefacts, mais aussi la mise en place d'un environnement de développement ou l'exécution de commandes au sein d'un environnement isolé. La reproductibilité bit à bit des builds, permise par Nix, constitue une propriété désirée pour le projet. Nix facilite en outre la mise en œuvre de la

cross-compilation, soit la compilation depuis une architecture CPU particulière vers une architecture différente, par exemple, de x86_64 vers ARM64.

L'ensemble de la chaîne de build du système est pris en charge par Nix : le noyau, les différentes crates composant le projet, ainsi que les artefacts finaux assemblés à partir de ces éléments. Le détail de cet assemblage est développé ultérieurement dans le [chapitre 3.10, Génération de l'image du système](#).

La compilation s'effectue via le compilateur nightly de Rust. Ce choix est motivé par le recours à la fonctionnalité `build-std`, permettant le build de la bibliothèque standard avec les mêmes options d'optimisation que le reste du projet (plutôt que d'utiliser une version pre-build), ce qui contribue à la minimisation de la taille des artefacts finaux, ainsi que par l'utilisation de règles de linting propres au canal nightly. Le qualificatif nightly, ou « unstable », désigne l'absence de garantie de pérennité des fonctions, options et autres éléments exposés par ce canal, ces derniers pouvant être modifiés ou retirés sans préavis. Cette instabilité porte uniquement sur la stabilité de l'interface exposée dans le temps, et non sur la fiabilité d'exécution de la fonctionnalité elle-même [42].

3.9. Configuration du noyau

La configuration du noyau repose sur la configuration par défaut propre à l'architecture cible, complétée par un ensemble d'options supplémentaires situées dans le fichier [linux/config/common.conf](#). Plusieurs fragments de configuration peuvent être fournis simultanément une fonction Nix est créée à cet effet, permettant de fusionner l'ensemble de ces fragments avec la configuration par défaut. Cette approche permet de ne stocker, dans le dépôt, que les changements apportés à la configuration par défaut, plutôt que la configuration complète.

Une commande est mise à disposition afin de faciliter la modification de cette configuration. Cette commande fusionne l'ensemble des fragments, ouvre l'interface `menuconfig` du noyau, puis calcule automatiquement, à la sortie de cette interface, la différence entre la configuration modifiée et trois références distinctes. La différence par rapport au `defconfig` est exposée sous `config.merged`, la différence par rapport à la configuration personnalisée sous `config.diff`, et la configuration complète sous `config.full`.

3.10. Génération de l'image du système

L'image finale du système, qu'il s'agisse d'une image ISO ou d'une image disque brute, est assemblée entièrement au moyen de Nix. Chaque crate Rust du projet correspond à une dérivation Nix l'ensemble de ces dérivations est regroupé dans un autre dérivation nommé `rootfsEnv`, lequel intègre également des binaires additionnels, tels que Podman. Cette dérivation génère un dossier regroupant l'ensemble de ces éléments au sein du `/nix/store`.

Une dérivation `rootfs` reprend cet environnement et y ajoute les liens symboliques nécessaires, de sorte que les répertoires `/bin`, `/etc`, etc., contiennent des liens symboliques pointant vers le `/nix/store`. Cette dérivation produit, en sortie, une image SquashFS. Deux dérivation additionnelles complètent cet assemblage : `kernel`, qui construit l'image du noyau selon les options de configuration retenues, et `initrd`, qui fournit le système de fichiers initial.

À partir de ces trois dérivation, `rootfs`, `initrd` et `kernel`, une dérivation `iso` assemble l'ensemble en une image ISO, exécutable via QEMU [43] ou sur un système physique. L'arbre de build est illustré dans la figure 12 :

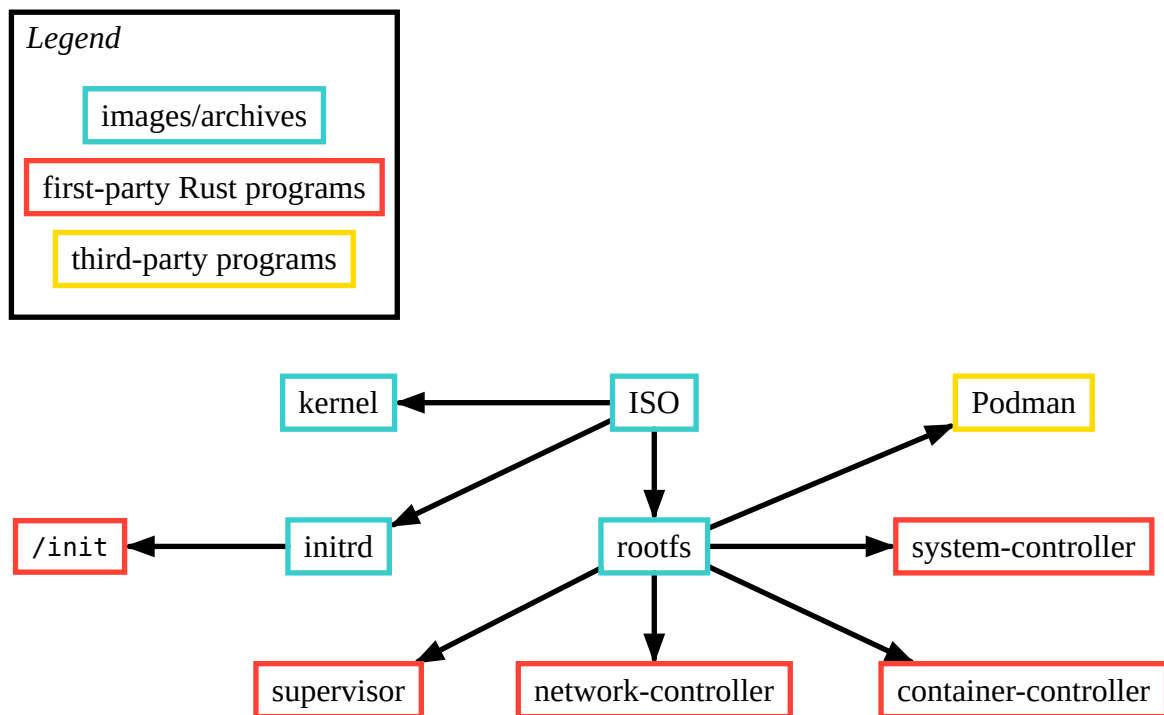


Figure 12. – Arbre de build

L'assemblage destiné à d'autres architectures suit une démarche similaire. Une image disque brute est par ailleurs requise pour les systèmes ne pouvant démarrer depuis une image ISO et nécessitant d'être flashés, tels que le Raspberry Pi. Une dérivation spécifique est créée pour chaque système visé par ce mode de déploiement. Dans le cas du Raspberry Pi, la dérivation `rpi-sd-image` regroupe les éléments propres à cette plateforme et produit une image directement destinée au flashage sur une carte SD.

3.11. Environnement de test

Trois catégories de tests sont mises en œuvre : les tests unitaires, les tests d'intégration, et les tests de bout en bout (end-to-end). Les tests unitaires sont dépourvus de complexité particulière, n'interagissant pas ou peu avec l'environnement extérieur. Ces tests sont réalisés à l'aide du système de test standard de Rust, via la macro `#[test]`.

Les tests d'intégration présentent une complexité supérieure. Le test du contrôleur réseau en constitue un exemple représentatif : une exécution directe sur l'hôte est exclue, la configuration réseau de ce dernier étant affectée, tandis qu'une exécution au sein du système complet introduirait de nombreux points de défaillance supplémentaires, certains éléments internes n'étant par ailleurs pas exposés à ce niveau. Une crate nommée `isolation`⁹ est créée à cet effet, permettant l'exécution d'un test dans un processus séparé, via un appel `fork` [44], isolé par des namespaces Linux [45]. Un environnement entièrement vierge est ainsi fourni pour chaque test, dépourvu d'interface réseau et disposant d'un système de fichiers racine propre. L'ensemble de cet environnement est détruit à l'issue du test. Cette isolation complète empêche cependant le test de binaires externes non présents dans l'environnement vierge ainsi constitué. Ces tests sont, comme les tests unitaires, réalisés via la macro `#[test]`, et localisés à proximité du code testé.

Les tests unitaires et les tests d'intégration partagent la propriété de ne pas nécessiter la reconstruction du système complet, ne reposant que sur l'écosystème Rust. Ces deux catégories de tests peuvent ainsi être exécutées directement au sein de l'environnement de développement Nix, la commande de test standard utilisant la compilation incrémentale de Rust plutôt qu'un rebuild complet.

Les tests de bout en bout exécutent le système complet ainsi qu'une suite d'actions et de mises à jour de configuration. Ces tests sont regroupés dans une crate dédiée, nommée `e2e`¹⁰. Cette catégorie de test vise à reproduire les conditions d'utilisation finale du système, ce qui nécessite l'exécution de l'image ISO complète. L'exécution de ces tests repose sur le lancement d'une machine virtuelle via QEMU, à partir de cette image ISO. Cette dépendance à l'image complète empêche l'exécution directe de ces tests via la commande de test standard de Rust : leur exécution passe par le système de build dans son intégralité.

L'ensemble des tests Rust est exécuté via l'outil Nextest [46]. Ce recours n'est pas strictement nécessaire pour les tests unitaires, mais s'avère requis pour les tests d'intégration. L'isolation propre à ces derniers provoque, en cas d'échec, un appel système `exit` [47] qui interrompt immédiatement le système de test standard de Rust, alors que Nextest poursuit son exécution, chaque test étant exécuté dans un thread distinct dont la terminaison est gérée indépendamment.

3.12. Pipeline CI/CD

L'ensemble du projet reposant sur Nix, le pipeline CI/CD ([.gitlab-ci.yml](#)) en hérite également, ce qui simplifie sa mise en œuvre : les étapes exécutées par le pipeline se limitent, dans une large mesure, à l'invocation de commandes Nix génériques, indépendantes de l'environnement d'exécution. L'ordonnancement des tâches repose sur GitLab CI [48]. Le recours à un runner personnalisé est requis, les runners institutionnels ne permettant ni l'utilisation de Nix ni la virtualisation imbriquée,

⁹Implémenté dans [rust/crates/isolation](#)

¹⁰Implémenté dans [rust/e2e](#)

requis pour l'exécution des tests de bout en bout, et disposant par ailleurs de ressources de calcul limitées.

Le pipeline est déclenché à chaque push sur le dépôt, entraînant l'exécution de l'ensemble des étapes suivantes, dans l'ordre. Une phase de linting est exécutée en premier lieu, portant à la fois sur la documentation et sur le code. Une phase de test lui succède, structurée séquentiellement : les tests unitaires et les doctests de Rust¹¹ sont exécutés en parallèle, suivis des tests d'intégration. La phase de build est ensuite exécutée, suivie enfin des tests de bout en bout. L'ensemble du processus est illustré dans la figure 13 :

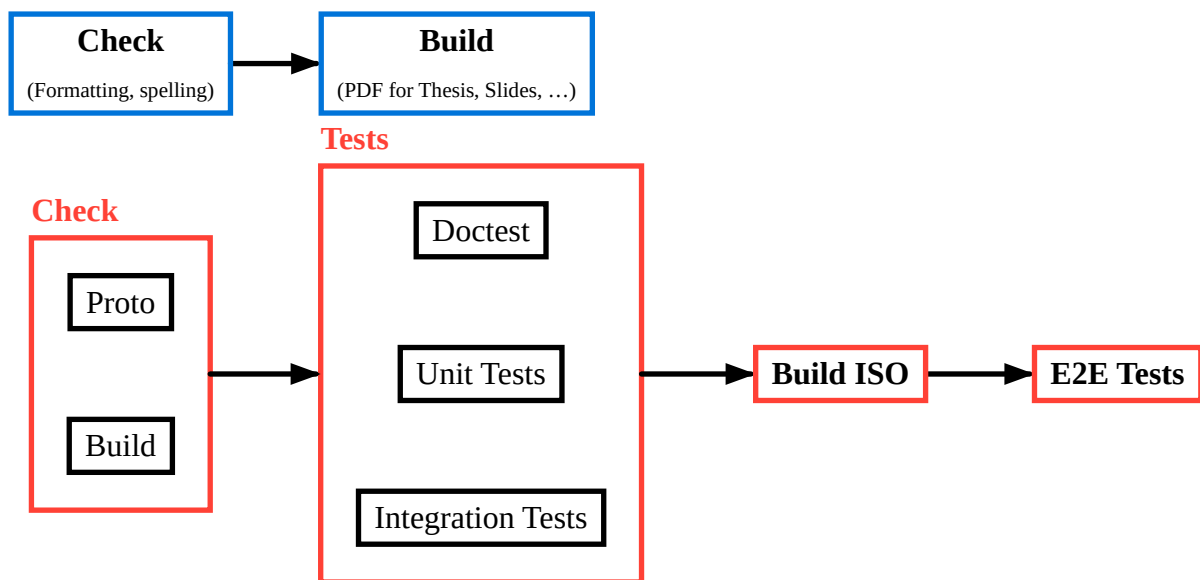


Figure 13. – Pipeline CI/CD

En bleu, le pipeline pour la documentation (thèse, slides, etc.). En rouge, le pipeline pour le code Rust. Les deux pipelines s'exécutent de manière indépendante.

Le linting couvre deux aspects distincts : le formatage et l'analyse statique. Le formatage est vérifié via `typstfmt` [50] pour la documentation, `buf` [51] pour les définitions Protocol Buffers, et l'outil natif pour le code Rust [52]. L'analyse statique repose aussi sur `buf` pour les définitions Protocol Buffers et sur `Clippy` [53] pour le code Rust. Les options les plus strictes de `Clippy` sont activées, interdisant notamment le recours à `unwrap` ou à des méthodes équivalentes, ainsi que l'utilisation de `println`, afin de garantir que toute sortie passe par le système de journalisation. Diverses règles additionnelles sont par ailleurs activées afin d'assurer l'homogénéité du code. Toute désactivation ponctuelle d'une règle doit être accompagnée d'une justification explicite, mécanisme nativement supporté par `Clippy` une exception dépourvue de justification est rejetée par le pipeline. Une exception générale est toutefois appliquée au code de test, pour lequel le recours à `unwrap` est autorisé, cette approche constituant la méthode recommandée pour exprimer une assertion dans ce contexte.

¹¹Un doctest est un exemple de code intégré à la documentation d'une fonction ou d'un module, compilé et exécuté automatiquement lors de l'exécution des tests afin de garantir la validité des exemples fournis [49].

Afin de simplifier l’invocation des commandes fréquemment utilisées, telles que le linting, Just [54], alternative au Makefile [55], est mis en place afin de définir des alias aux commandes fréquemment utilisées, telles que le linting ou l’exécution des tests.

Chapitre 4

Tests et validation

Ce chapitre présente la démarche de validation adoptée pour vérifier le comportement du système implémenté. La couverture assurée par les tests unitaires et les tests d'intégration, propres à chaque composant, est d'abord présentée, avant que les tests de bout en bout, reproduisant des scénarios d'utilisation complets du système, ne soient détaillés. Ces derniers servent également de base à l'analyse de performance, qui mesure la rapidité de démarrage et d'installation du système ainsi que son empreinte mémoire. Le chapitre se conclut par une discussion des limites du protocole de mesure employé.

4.1. Tests unitaires et tests d'intégration

Les tests unitaires et les tests d'intégration portent principalement sur les contrôleurs. La logique de validation des ressources est systématiquement couverte par des tests unitaires. Les tests d'intégration couvrent systématiquement les cas suivants : la création d'une nouvelle ressource, la mise à jour d'une ressource existante, la tentative de gestion d'une ressource inexistante (par exemple une interface réseau absente), la tentative de gestion d'une ressource déjà gérée par un autre composant, ainsi que la suppression d'une ressource. Seule une partie des contrôleurs sont testés via des tests d'intégration. En effet, l'environnement dans lequel les tests sont isolés ne dispose pas de tous les éléments nécessaires pour mener à bien le test de certains composants. C'est notamment le cas du contrôleur de conteneurs : l'environnement de test disposant d'un système de fichier racine vide, le runtime de conteneur n'est pas présent et celui-ci nécessite un nombre important de dépendances qu'il serait fastidieux de lier dans cet environnement.

Toutefois, des tests de bout en bout (end-to-end, E2E), implémentés dans [rust/e2e/tests/](#) viennent compléter la couverture. Ces tests reposent fortement sur Nix afin de créer les artefacts permettant d'exécuter une machine virtuelle, en particulier une image ISO. Chaque test est effectué dans une machine virtuelle séparée, tel que présenté dans le [chapitre 3.11, Environnement de test](#). De manière générale, le test va lancer la machine virtuelle, attendre que l'API soit joignable, puis effectuer une suite de commandes. Afin de garantir que le système fonctionne correctement, les tests ne reposent pas uniquement sur la lecture de l'état courant de l'API, mais incluent dans la configuration un conteneur cURL qui va effectuer une requête HTTP vers l'hôte de test, sur un port que le test écoute. La réception de cette requête permet ainsi de valider que l'état rapporté par l'API ne se contente pas d'être cohérent en apparence, mais reflète bien l'état réel du système et démontre que le conteneur fonctionne.

4.2. Validation

Parmi les tests de bout en bout, trois scénarios notables sont définis : l'exécution d'un conteneur dans un environnement éphémère, l'installation du système suivi de l'exécution d'un conteneur, et le déploiement d'une application 3 tiers.

4.2.1. Exécution dans un environnement éphémère

La configuration est appliquée directement sur le système démarré depuis l'image ISO, sans écriture sur disque, et définit un unique conteneur exécutant une requête HTTP vers un service exposé par l'hôte de test. La validation consiste à vérifier que l'état rapporté par l'API à l'issue du cycle de démarrage correspond à l'état attendu (conteneur en cours d'exécution), et que la requête HTTP émise par le conteneur est effectivement reçue par l'hôte de test.

4.2.2. Installation du système

Le système est installé sur le disque de la machine virtuelle à partir de l'image ISO, puis redémarré. La configuration appliquée définit également un conteneur, dont l'exécution est vérifiée selon les mêmes critères que ceux du scénario précédent. La validation porte en outre sur la persistance de l'état à travers le redémarrage : l'état de la configuration et du conteneur, tel que rapporté par l'API après redémarrage, doit correspondre à l'état appliqué avant redémarrage.

4.2.3. Application 3 tiers

Ce scénario met en œuvre une configuration composée de quatre conteneurs : une base de données, un service backend dépendant de la base de données, un service web dépendant du backend, et un conteneur de « probe » dépendant du service web, chargé d'émettre une requête HTTP l'hôte.

Lorsque l'hôte reçoit la requête HTTP, le test va alors initier une requête sur le conteneur « web » qui va transmettre celle-ci au conteneur « backend », puis la persister sur la base de données. Le résultat est ensuite vérifié, puis la machine redémarrée, puis le résultat revérifié afin de valider que la donnée a bien été persister.

4.3. Analyse de performance

Les scénarios d'exécution en environnement éphémère et d'installation du système sont repris ici selon le même protocole, en y ajoutant une instrumentation permettant de mesurer le temps écoulé entre chaque étape du cycle de vie, ainsi que la mémoire consommée par le système, le tout sur 100 échantillons.

Les mesures reposent sur la configuration QEMU présentée au code 21, exécutée sur un hôte doté d'un processeur AMD Ryzen 7 7700X, de 64 GiB de mémoire vive et d'un support de stockage NVMe. Cet hôte fonctionne sous Windows 11, les tests étant effectués au sein d'un environnement WSL 2 exécutant Debian.

```
1 qemu-system-x86_64 -cdrom result \
2   -drive file=disk.img,format=raw,if=virtio \
3   -enable-kvm \
4   -cpu host -m 256M \
5   -netdev user,id=net0,hostfwd=tcp::50000-:50000 \
6   -device e1000,netdev=net0 \
7   -nographic
```

Code 21. – Commande QEMU utilisée pour les mesures de performance

La majorité des instants mesurés sont déterminés par horodatage, côté tests, des messages émis sur la console par les différents composants du système, cette dernière étant flushée immédiatement après chaque message afin de garantir la fidélité de l’horodatage par rapport à l’instant d’émission. La marge d’erreur associée à ce mécanisme est jugée négligeable au regard de l’échelle de temps mesurée. Le démarrage d’un conteneur constitue une exception à ce mécanisme : cette mesure repose sur le même protocole que les tests de bout en bout décrits précédemment, à savoir la réception, par un serveur à l’écoute sur l’hôte, d’une requête HTTP émise par le conteneur concerné.

4.3.1. Rapidité

La figure 14 présente la chronologie des étapes de démarrage jusqu’à l’exécution d’un conteneur, pour deux modes de démarrage : une installation préalable sur disque (plan supérieur) et un démarrage éphémère depuis l’image ISO, sans installation (plan inférieur). Cinq instants sont mesurés depuis le démarrage du noyau par le bootloader : le passage à /init (« Time until /init », en vert ) , la réception d’une route via DHCP (« Time until DHCP route received », en bleu ) , moment à partir duquel l’API devient accessible, le début du téléchargement d’une image de conteneur (« Time until image downloading », en turquoise ) , le démarrage d’un conteneur dont l’image est déjà présente localement (« Time until container started (no pull) », en violet ) , et le démarrage d’un conteneur dont l’image doit être téléchargée (« Time until container started (pull) », en rouge ) :

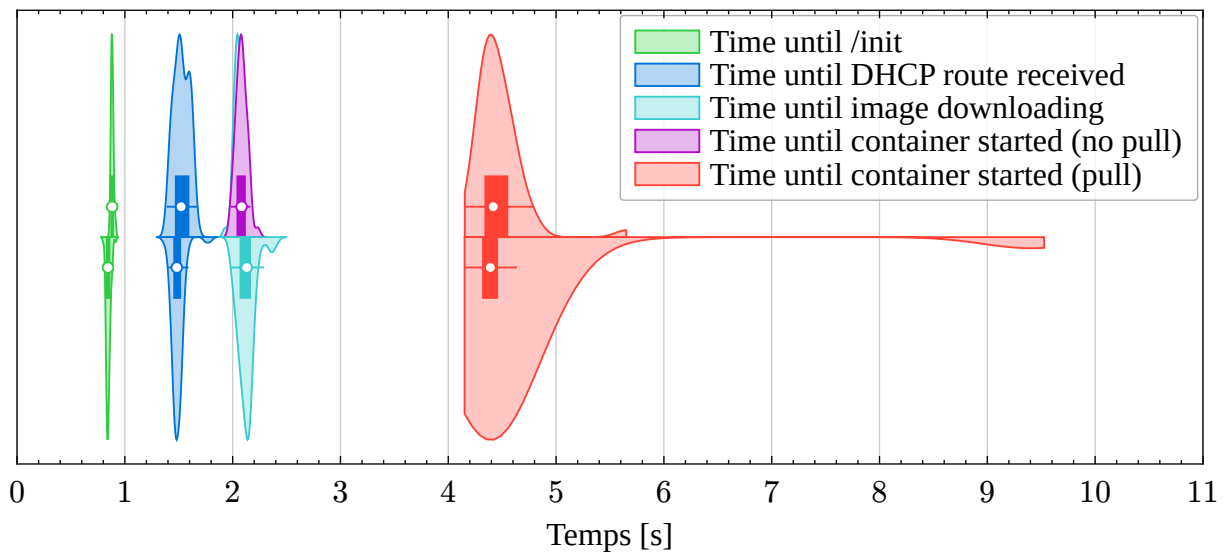


Figure 14. – Chronologie des étapes de démarrage jusqu'à l'exécution d'un conteneur

Temps absolu en secondes mesuré à partir du démarrage du noyau, après le bootloader, des différentes étapes du cycle de vie sur un échantillon de 100.

La mesures depuis une installation sur disque sont représentées sur le plan supérieur, tandis que les mesures depuis l'image ISO (mode éphémère) sont représentées sur le plan inférieur.

Dans les deux modes de démarrage, un peu moins d'une seconde s'écoule entre le démarrage du noyau et le passage à `/init`, puis environ 0.6 seconde supplémentaire est nécessaire à la réconciliation et au protocole DHCP, portant à environ 1.5 seconde le délai avant que l'API ne devienne joignable.

Le téléchargement de l'image du conteneur, s'il y a lieu, commence environ 0.5 seconde après la configuration DHCP. Une fois celui-ci commencé, environ 2.1 secondes sont nécessaires pour qu'il arrive à son terme. Le conteneur est immédiatement démarré une fois ce téléchargement terminé. Lorsque l'image est déjà téléchargée, le téléchargement se termine instantanément et le conteneur est aussitôt démarré, ce qui crée une superposition des deux événements sur la figure 14. Le cas d'une image déjà téléchargée n'est, par nature, pas possible pour un environnement éphémère et n'est donc pas représenté sur le plan inférieur.

Au total, entre le démarrage de la machine et le démarrage du conteneur, le temps médian est de 5.1 secondes dans le cas d'un téléchargement d'image, contre environ 2.1 secondes lorsque l'image est déjà présente localement. En excluant les délais qui ne relèvent pas directement du système, à savoir le chargement du noyau, l'obtention d'une configuration réseau via DHCP et le téléchargement de l'image, le temps propre au démarrage du conteneur est inférieur à 500 millisecondes. Il n'y a par ailleurs pas de différence notable entre un démarrage sur disque et un démarrage depuis l'image ISO.

La figure 15 présente la durée du processus d'installation (« Time to install », en bleu ) , ainsi que la durée totale jusqu'au démarrage d'un conteneur après installation (« Time until container started », en orange ). La première mesure l'intervalle entre la réception de la configuration et la fin

de l'écriture des artefacts sur le disque cible, avant redémarrage. La seconde mesure l'intervalle entre ce même instant de référence et le démarrage du conteneur, incluant le redémarrage du système, la réconciliation réseau et le téléchargement de l'image :

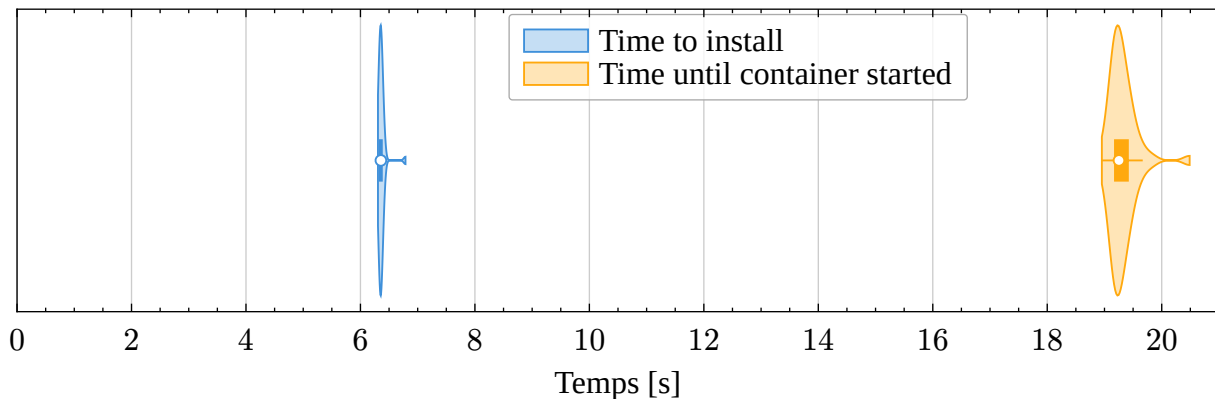


Figure 15. – Durée d'installation et de démarrage du conteneur après installation
Temps absolu en secondes mesuré à partir de la réception de la configuration jusqu'à
l'installation du système, et temps entre le démarrage du noyau et l'exécution d'un conteneur,
installation incluse, sur un échantillon de 100.

L'installation proprement dite se conclut en environ 6.3 secondes. Le démarrage complet du conteneur, incluant le redémarrage du système et le cycle décrit à la figure 14, se conclut quant à lui en environ 19.3 secondes. Ce total inclut le temps nécessaire à l'hyperviseur pour redémarrer la machine virtuelle (par exemple le chargement du BIOS) et le délai de sélection du bootloader (environ 5 secondes).

4.3.2. Légèreté

Une machine virtuelle disposant de 256 MiB de RAM est utilisée pour ce test. Une configuration minimale est appliquée en mode éphémère, définissant un conteneur nommé chargé d'exécuter en boucle la séquence suivante : allocation d'un vecteur, écriture d'une suite de valeurs dans ce vecteur, vérification de la suite, envoi de la taille d'allocation courante via un socket, désallocation, puis incrémentation de l'allocation d'un mégaoctet. Cette boucle se poursuit jusqu'à l'échec de l'allocation ou de la vérification, ce qui permet de déterminer la quantité de mémoire effectivement disponible pour un conteneur une fois le système démarré.

La figure 16 présente la distribution de l'allocation mémoire maximale atteinte par le conteneur sur cent exécutions, une fois le système démarré :

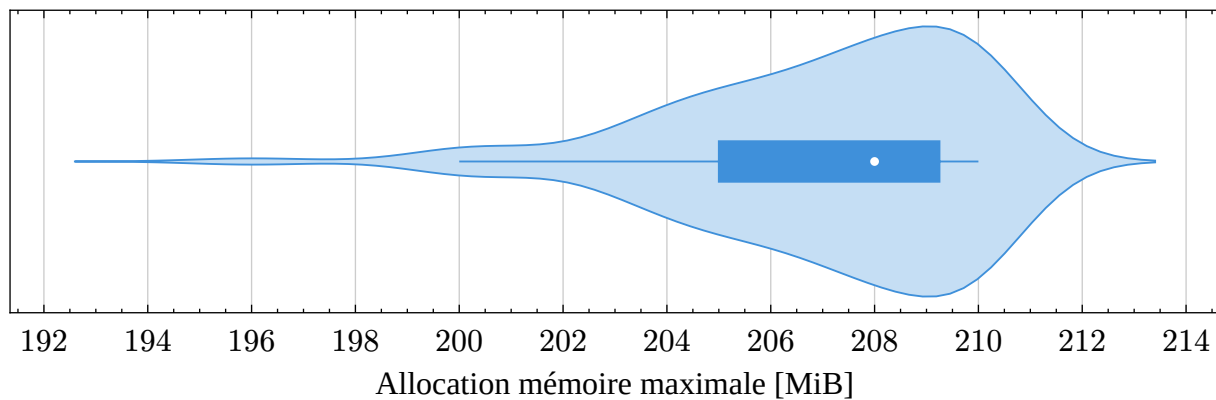


Figure 16. – Mémoire disponible sur le système

Allocation mémoire maximale faisable par un conteneur sur une VM disposant de 256 MiB sur 100 échantillons.

La médiane de l'allocation atteinte se situe à environ 208 MiB. Les valeurs extrêmes observées se situent entre 193 et 213 MiB. La mémoire disponible est donc autour des 80% de la mémoire allouée à la machine virtuelle. Par extension, le système d'exploitation complet, noyau et runtime de conteneur inclus, ne consomme donc qu'environ 40 MiB. Toutefois, il n'est pas pour autant possible de démarrer une machine virtuelle avec seulement 20 MiB de mémoire. En effet, durant le démarrage, un minimum de 80 MiB est requis afin que le système démarre, et dans l'optique de télécharger une image et exécuter un conteneur, au minimum 160 MiB sont requis.

4.4. Exécution sur un Raspberry Pi 1B

Le Raspberry Pi 1B est l'un des premiers modèles de Raspberry Pi : il repose sur une puce BCM2835 comportant 512 MiB de mémoire vive et un CPU ARM1176JZF-S, sous architecture ARMv6. La famille de CPU ARM11 est commercialisée entre 2002 et 2005, ce qui témoigne de l'ancienneté de la plateforme. Cette dernière constitue de ce fait une limite matérielle extrême sur laquelle valider la solution.

S'agissant d'un matériel physique, les tests sont exécutés manuellement sur le Raspberry Pi et les mesures sont donc approximatives. Les trois scénarios notables décrits au [chapitre 4.2, Validation](#) sont testés sans qu'aucune adaptation particulière ne soit nécessaire¹². L'ensemble des tests réussit, avec toutefois une durée sensiblement plus longue : les ressources limitées du Raspberry Pi allongent considérablement chaque étape: dans le scénario d'exécution dans un environnement éphémère, l'exécution complète prend environ 38 secondes. Une grande partie du temps est passé dans la réception d'une route DHCP et dans le téléchargement de l'image. En outre, la création à proprement dite du conteneur prend également un temps considérable (environ 4 secondes).

¹²Le Raspberry Pi ne pouvant être démarré depuis un support externe autre que le lecteur de carte SD intégré, le système est considéré comme déjà installé.

4.5. Limitations

Le protocole de mesure employé pour les benchmarks de rapidité et de légèreté ne reflète pas nécessairement l'ensemble des conditions rencontrées en usage réel. Les mesures présentées correspondent à un scénario favorable, dans lequel le délai d'obtention d'une adresse via DHCP, la charge du processeur hôte et la latence réseau ne sont pas artificiellement dégradés. Une charge processeur ou un délai réseau plus élevé que ceux observés durant les mesures conduiraient à une augmentation des temps rapportés, notamment pour les étapes dépendant du réseau, telles que la réconciliation DHCP et le téléchargement d'image.

Chapitre 5

Comparaison avec les solutions existantes

Ce chapitre compare directement le système implémenté aux deux solutions les plus proches identifiées durant le projet de semestre, à savoir NixOS et Talos Linux. Les critères d'évaluation retenus sont d'abord définis, avant que la méthodologie de mesure ne soit précisée et que chaque solution ne soit présentée et évaluée individuellement selon ces critères. Une synthèse comparative conclut ce chapitre.

5.1. Critères

Les critères utilisés sont repris du projet de semestre [56].

5.1.1. Automatisation

Le cycle de vie de la solution doit être entièrement automatisable, dès la phase d'installation. Plus spécifiquement, l'installation, la configuration initiale et le déploiement de l'hôte doivent pouvoir être réalisés à distance, de manière programmatique et sans intervention humaine. L'installation et la configuration du système doivent en outre reposer sur les mêmes commandes, sans procédure dédiée à la seule phase d'installation.

5.1.2. Légèreté

La solution doit pouvoir fonctionner avec une quantité de mémoire vive limitée, l'objectif visé étant un fonctionnement stable avec 256 MiB de RAM, et la limite acceptable étant fixée à 512 MiB. Le critère est considéré comme atteint uniquement si la limite de 512 MiB est respectée à la fois lors de l'installation et lors de l'exécution.

5.1.3. Rapidité

La solution doit permettre un déploiement rapide, aussi bien lors d'une installation initiale que lors d'un démarrage ultérieur du système. Ce critère est évalué au moyen de deux mesures : le temps d'installation, mesuré entre l'entrée dans l'environnement d'installation et le démarrage du conteneur configuré, et le temps de démarrage, mesuré entre le lancement de la machine virtuelle et le démarrage de ce même conteneur.

5.1.4. Simplicité

La solution doit être aussi simple que possible à l'utilisation. La configuration doit être centralisée, clairement structurée et facilement modifiable par une personne ayant des connaissances dans le domaine concerné.

5.2. Méthodologie

Pour chaque solution évaluée, la configuration retenue correspond à la configuration par défaut recommandée par la documentation officielle du projet, sans optimisation spécifique.

La rapidité est mesurée au moyen d'un conteneur effectuant une requête HTTP vers l'hôte dès son démarrage, selon la même procédure que celle décrite au [chapitre 4.3, Analyse de performance](#) : l'instant de réception de cette requête par l'hôte marque la fin de la mesure, aussi bien pour le temps d'installation que pour le temps de démarrage.

La légèreté est évaluée à partir de ce même protocole de mesure de la rapidité : une machine virtuelle est démarrée avec une quantité de mémoire vive donnée, puis la requête HTTP attendue est surveillée pendant une durée maximale de cinq minutes. L'absence de réception de cette requête dans ce délai est interprétée comme un échec du démarrage du conteneur pour la quantité de mémoire testée. En cas d'échec, une nouvelle tentative sera effectuée avec plus de mémoire vive jusqu'à ce que cela fonctionne.

5.3. NixOS

NixOS est une distribution Linux généraliste construite autour du gestionnaire de paquets Nix, dont le périmètre couvre l'ensemble des usages d'un système d'exploitation traditionnel, la gestion de conteneurs n'en constituant qu'un usage parmi d'autres. La configuration du système repose sur un langage déclaratif et fonctionnel, permettant une forte reproductibilité des installations ainsi que des mises à jour atomiques. L'installation d'un système NixOS s'effectue traditionnellement à partir d'un support d'installation exécutant un installeur interactif, la configuration déclarative n'intervenant qu'une fois le système de base installé.

L'évaluation de la solution est réalisée sur la version 25.11, qui utilise la version 6.12.62 du noyau Linux, publiée le 30 novembre 2025. Cette version est disponible sous la licence MIT. La configuration utilisée pour cette évaluation est disponible dans [misc/nixos/vm.nix](#). La mémoire requise s'élève à 276 MiB en exécution et à 762 MiB à l'installation. Le temps d'installation mesuré est de 5 minutes, et le temps de démarrage de 31 secondes.

Le critère d'automatisation n'est que partiellement atteint : l'installation de NixOS nécessite un installeur interactif ou un mécanisme d'initialisation externe (par exemple `nixos-anywhere` [57]) chargé d'effectuer l'installation initiale, cette étape reposant ainsi sur des commandes distinctes de celles utilisées pour la configuration déclarative subséquente du système. Le critère de légèreté n'est par ailleurs pas atteint, la mémoire requise à l'installation (762 MiB) dépassant la limite acceptable de 512 MiB, bien que la mémoire requise en exécution seule (276 MiB) reste dans à cette limite.

5.4. Talos Linux

Talos Linux est une distribution Linux immuable et minimale, développée par Sidero Labs et conçue exclusivement pour l’exécution de clusters Kubernetes. Le système de fichiers racine est en lecture seule et l’ensemble de l’administration s’effectue via une API dédiée. La configuration du système est décrite intégralement par un unique fichier déclaratif, appliqué dès le premier démarrage de la machine.

L’évaluation porte sur la version 1.11.5, qui utilise la version 6.12.52 du noyau Linux, publiée le 6 novembre 2025. Ce logiciel est distribué sous la licence MPL-2.0. La configuration utilisée pour cette évaluation est disponible dans [misc/talos/config.yaml](#). La mémoire requise s’élève à 1.4 GiB, aussi bien à l’installation qu’en exécution. Le temps d’installation mesuré est de 210 secondes, et le temps de démarrage de 65 secondes.

Le critère d’automatisation est atteint : la même configuration déclarative, transmise via l’API du système, est utilisée aussi bien pour l’installation initiale que pour toute reconfiguration ultérieure, sans qu’aucune commande distincte ne soit requise entre ces deux phases. Le critère de légèreté n’est en revanche pas atteint, la mémoire requise (1.4 GiB) dépassant très largement la limite acceptable de 512 MiB, aussi bien à l’installation qu’en exécution.

5.5. Synthèse

Le tableau 4 rassemble, pour les trois systèmes évalués, les résultats obtenus pour chacun des critères définis en début de chapitre :

Critères	ContainerOS	NixOS	Talos
Dernière date	∅	2025-11-30	2025-11-06
Version évaluée	∅	25.11	1.11.5
Version du noyau	6.18.11	6.12.62	6.12.52
Licence	∅	MIT	MPL-2.0
Automatisation	✓	★	✓
Mémoire requise en exécution	160 MiB	276 MiB	1.4 GiB
Mémoire requise à l’installation	160 MiB	762 MiB	1.4 GiB
Rapidité d’installation	36s	300s	210s
Rapidité de démarrage	5.6s	31s	65s
Simplicité	✓	✓	✗

Tableau 4. – Comparaison des solutions existantes

- ✓ signifie que le critère est atteint
- ✗ signifie que le critère n’est pas atteint
- ★ signifie que le critère n’est que partiellement atteint
- ∅ signifie que la valeur n’est pas applicable

La solution développe, ContainerOS, est la seule des trois solutions à satisfaire l'ensemble des critères retenus. NixOS échoue le critère de légèreté à l'installation, sa consommation mémoire dépassant alors la limite acceptable malgré une empreinte en exécution conforme, ainsi que le critère d'automatisation, en raison de la dépendance à un installeur interactif ou à un mécanisme d'initialisation externe distinct de la configuration déclarative elle-même. Talos Linux échoue quant à lui le critère de légèreté de manière constante, aussi bien à l'installation qu'en exécution, ainsi que le critère de simplicité, son périmètre étant orienté vers la gestion de clusters Kubernetes plutôt que vers le déploiement unitaire visé. Sur le plan de la légèreté, ContainerOS requiert environ deux fois moins de mémoire que NixOS et près de dix fois moins que Talos Linux, tout en restant dans les deux cas sous la limite acceptable de 512 MiB, y compris sur une machine dotée de seulement 256 MiB de RAM. Sur le plan de la rapidité, ContainerOS démarre environ cinq fois plus vite que NixOS et près de dix fois plus vite que Talos Linux le temps d'installation est quant à lui inférieur d'un facteur proche de dix par rapport à ces deux mêmes solutions.

Chapitre 6

Résultats et Discussion

Ce chapitre dresse le bilan du travail réalisé. Les résultats obtenus sont d’abord présentés, tant sur le plan fonctionnel qu’au regard des objectifs académiques fixés en introduction, avant qu’un retour rétrospectif ne soit fait sur les principaux choix techniques effectués. Les difficultés rencontrées au cours du développement sont ensuite détaillées, avant que le chapitre ne se conclue par les perspectives d’évolution envisageables pour le système.

6.1. Résultats fonctionnels

Les scénarios de validation présentés au [chapitre 4, Tests et validation](#) démontrent que le système fonctionne bien dans trois configurations représentatives, et la comparaison avec NixOS et Talos Linux établit que le système présente une empreinte mémoire et des performances nettement meilleures.

Le système fonctionne également sur un Raspberry Pi 1B, sans qu’aucune adaptation notable n’ait été nécessaire pour ce matériel. Bien que le projet visait à être déployé sur des Raspberry Pi modernes, la capacité du système à fonctionner sur un modèle aussi ancien tout en restant relativement rapide illustre concrètement l’atteinte de l’objectif de légèreté, ce matériel disposant de ressources sensiblement inférieures à celles des modèles visés initialement.

D’un point de vue fonctionnel, la valeur ajoutée du système réside dans sa rapidité de démarrage et dans sa faible empreinte mémoire, deux propriétés mesurées au [chapitre 4.3, Analyse de performance](#) et confirmées par la comparaison avec les solutions existantes : le système démarre un conteneur en quelques secondes et fonctionne avec une empreinte mémoire de 160 MiB, ce qui permet un déploiement quasi immédiat, aussi bien depuis une image ISO que depuis une installation sur disque.

Le système présente toutefois des limites fonctionnelles. Deux fonctionnalités identifiées lors de la planification ne sont pas implémentées : la gestion de tâches planifiées (« job scheduling ») ainsi que le monitoring du système. L’absence de tâches planifiées fait qu’il est plus compliqué d’exécuter des routines régulières (par exemple des sauvegardes), tandis que l’absence de monitoring complique la détection et le diagnostic d’une défaillance. Ces deux limitations restent toutefois partiellement contournables dans l’état actuel du système, ces deux fonctionnalités pouvant elles-mêmes être

déployées sous forme de conteneurs. En outre, seule une partie des options de configuration du réseau et des conteneurs a été implémentée, principalement dans un souci de temps.

6.2. Résultats académiques

L'ensemble des objectifs techniques formulés dans l'énoncé du sujet sont remplis par le système développé. Les résultats fonctionnels présentés au [chapitre 4, Tests et validation](#), en particulier la validation des trois scénarios de bout en bout et les mesures effectuées confirment cela. Les deux fonctionnalités non implémentées ne figurent pas parmi les objectifs centraux de l'énoncé et sont écartées afin de concentrer l'effort sur la robustesse du système, jugée prioritaire. Au regard de l'ensemble de ces éléments, les objectifs de l'énoncé du sujet sont considérés comme atteints avec grande satisfaction.

La planification initiale du travail de diplôme est établie sous la forme d'un diagramme de Gantt, structuré par semaine sur l'ensemble de la durée du projet. La figure 17 présente cette planification, complétée a posteriori par l'avancement effectif de chaque tâche.

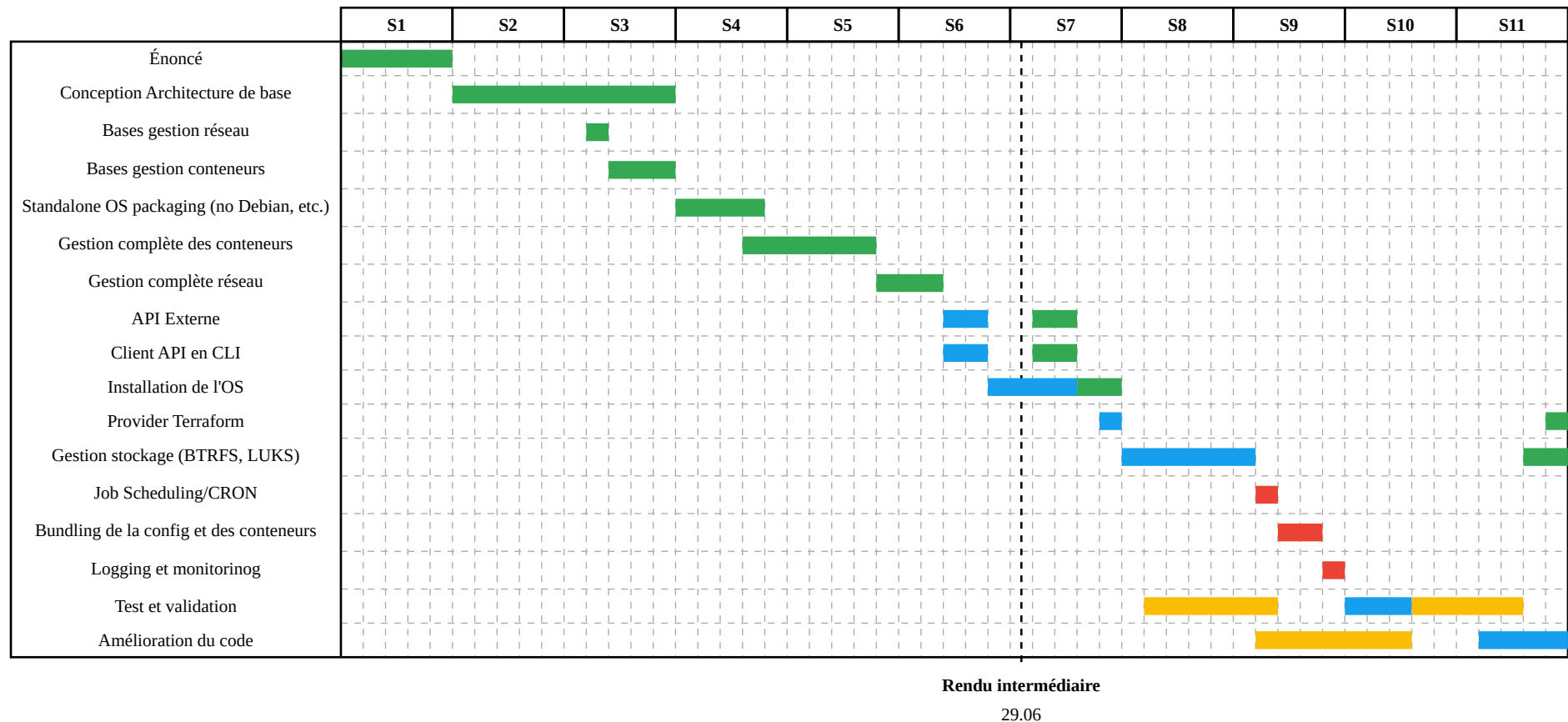


Figure 17. – Planification du travail

Planification du travail sous la forme d'un diagramme de Gantt. En vert les tâches dont la planification de la durée est correcte. Si elles n'ont pas été faites au moment prévu, le moment prévu est superposé en bleu. En rouge les tâches non effectuées. En orange, les tâches ayant duré plus longtemps que prévu.

La planification reste fidèle à l'avancement effectif jusqu'au début du mois de juillet. À partir de cette période, la correspondance entre la planification et l'avancement réel devient plus approximative. La tâche « Test et validation » figurant sur le diagramme ne recouvre pas l'ensemble des activités de test menées durant le projet, le système faisant l'objet de tests continus dès les premières étapes de son développement, notamment au moyen des tests unitaires et d'intégration décrits au [chapitre 4, Tests et validation](#). Cette tâche est avancée et prolongée au-delà de la durée prévue, recouvrant une partie de la période initialement réservée à l'amélioration du code, cette dernière tâche ayant elle-même été anticipée. Ce chevauchement s'explique par la nécessité de consolider certains composants avant que les tests de bout en bout ne puissent être exécutés de manière fiable.

6.3. Difficultés rencontrées

Le développement du système, bien qu'ayant abouti à une solution fonctionnelle, conforme aux objectifs fixés, et satisfaisante, ne s'est pas déroulé sans obstacle. Plusieurs difficultés techniques sont rencontrées durant le développement du système. Certaines relèvent de particularités peu documentées d'un composant, d'autres de bugs identifiés dans des dépendances externes, et d'autres encore de contraintes propres à l'environnement de développement utilisé.

Un comportement inattendu est observé lors de la mise en place de l'initrd nécessaire au démarrage du système. Lorsque la mémoire disponible est insuffisante, l'initrd n'est que partiellement chargé, ce qui introduit une confusion importante lors du diagnostic : le processus `/init` s'exécute normalement et parvient à lire certains fichiers, tandis que d'autres fichiers, censés être présents, sont rapportés comme inexistants. Ce mode de défaillance silencieux est surprenant, d'autant plus qu'aucune information affichée sur la console par le noyau ne laissait supposer un tel problème.

Un bug est également identifié dans `smoltcp`, seule bibliothèque Rust utilisable comme client DHCP dans le contexte de ce système. La bibliothèque standard de Rust fournit plusieurs types qui sont réutilisés à travers tout l'écosystème, mais certaines, dont `smoltcp`, réimplémentent leurs propres types, notamment un équivalent de `std::time::Instant`. Une interface `From` est alors fournie pour convertir un type vers l'autre : dans le cas de `smoltcp`, cette conversion est incorrectement implémentée et retourne systématiquement l'instant présent, faussant l'ensemble des calculs de délai reposant sur cette conversion [58]. Le bug c'est avérer trivialement réglable et, étant donné qu'il s'agit d'un élément essentiel du système, et compte tenu de l'absence d'autres alternatives, un correctif permettant de régler ce bogue a été créé, soumis via une pull request, et accepté, sur le dépôt de `smoltcp` [59].

Un bug est aussi rencontré dans Podman, version 5.8.4. Podman revendique une compatibilité « drop-in » avec Docker, sans que cette compatibilité soit exacte en pratique. La fonction `list_containers` retourne notamment une structure légèrement différente : Docker rapporte un état `"status": "exited"` associé au champ `"stopped": true`, alors que Podman rapporte, pour un état équivalent, la valeur `"stopped"` pour ce même champ `status`. Ce bug est connu depuis 2023 [60]

et n'est corrigé par le projet qu'en mars 2026 [61], la correction n'étant intégrée à une version officielle qu'avec la publication de la version 6.0.0, le 24 juin 2026 [62]. L'intégration de cette version dans Nix, celle-ci constituant une « breaking release », nécessite un délai supplémentaire et demeure bloquée [63]. Le paquet Nix officiel est alors repris et adapté directement à partir de son code source, l'aspect « breaking » de la version 6.0 de Podman n'ayant pas d'impact dans le cadre de ce système.

Au-delà de ce bug, Podman présente une autre particularité : son fonctionnement sans daemon (« daemon-less ») ne s'applique qu'à l'utilisation de la ligne de commande podman. Aucune interface native (« bindings ») n'est fournie pour une intégration directe. Podman doit donc être exécuté comme une API, avec laquelle une interaction s'effectue via HTTP ou TCP, ce qui revient, en pratique, à le traiter comme un daemon.

Un bug est par ailleurs observé sous WSL, environnement utilisé pour une partie du développement. La création d'un fichier temporaire via `open2` et l'option `O_TMPFILE`, suivie d'une tentative de le rendre permanent via `linkat`, produit une erreur indiquant que le fichier n'existe pas. Ce bug affecte les tests d'intégration du contrôleur système, responsable de la gestion des fichiers : ces tests échouent sous WSL, mais s'exécutent correctement sur une machine Debian standard.

Le temps de build de Nix constitue une autre difficulté notable. Nix isole intégralement chaque build et ne tire volontairement pas parti de la compilation incrémentale, ce qui pénalise particulièrement Rust, dont les temps de compilation sont réputés élevés. Un build complet de la partie userspace en Rust peut atteindre 5 minutes, sans compter le temps de build du noyau, lui-même considérable même sur une machine puissante. La bibliothèque communautaire « crane » permet de mettre en cache une partie des dépendances externes, mais sa documentation reste limitée et son usage impose une structuration précise du projet.

6.4. Rétrospective

Le choix de Rust et de Nix constitue, malgré les difficultés rencontrées, un choix technique approprié. Le système présente une robustesse satisfaisante, au sens où une erreur ne provoque pas d'arrêt inattendu du système, et la reproductibilité des builds constitue un avantage important dans un contexte de développement réparti sur plusieurs appareils et intégré à une pipeline CI/CD.

Le choix architectural d'un modèle centralisé est plus nuancé. Ce modèle simplifie effectivement l'ordonnancement, objectif initial de ce choix, mais réduit la flexibilité du système, désavantage identifié dès la phase de conception. L'implémentation des différents contrôleurs révèle des besoins particuliers à chacun, ce qui n'est pas imputable au modèle centralisé en tant que tel, mais complexifie le composant central à mesure que les cas d'usage se diversifient. Ce choix est considéré comme neutre au regard du compromis obtenu.

Le choix de gRPC pour la communication entre le contrôleur central et les contrôleurs répond avant tout à un besoin de rapidité de mise en œuvre. Cette communication reste, dans une certaine mesure, lourde à maintenir, bien que cette lourdeur soit largement masquée pour l'utilisateur final. Un protocole custom, basé sur des sockets UNIX, constituerait potentiellement une alternative plus adaptée, mais le système actuel fonctionne de manière satisfaisante, rendant cette amélioration secondaire.

6.5. Perspectives

Le système répond, dans son périmètre actuel, aux objectifs fixés, mais son architecture n'exclut pas des évolutions ultérieures. Plusieurs perspectives d'évolution se dégagent de ce travail, tant dans le périmètre initial que dans des directions plus larges.

Concernant l'usage primaire du système, centré sur les conteneurs, trois fonctionnalités apparaissent prioritaires : l'ajout d'une couche d'observabilité, l'ajout d'un mécanisme de planification de tâches similaire à CRON, et la gestion plus poussée du stockage. D'autres extensions sont envisageables dans un second temps, notamment l'extension de la gestion du réseau, par exemple pour le support des VPN ou du routage plus complexe, ainsi que l'extension de la gestion des conteneurs avec un modèle similaire aux « Pods » de Kubernetes ou de Podman.

Le projet peut également être rendu plus générique. La partie orchestration et ordonnancement est conceptuellement découplée des ressources qu'elle gère, ce qui ouvre la voie à une bibliothèque générique. La gestion du réseau, en particulier, réimplémente en grande partie des fonctionnalités déjà couvertes par systemd-networkd ou Netplan : cette partie pourrait être extraite sous forme de composant externe qui serait à la fois une alternative à ces deux solutions, tout en étant une partie intégrale du système.

Le système peut aussi être étendu pour supporter la virtualisation via libvirt, l'orchestrateur ne percevant, dans tous les cas, qu'une ressource à réconcilier, le comportement spécifique restant délégué au contrôleur correspondant. Cela nécessiterait toutefois d'implémenter divers aspects supplémentaires pour permettre de déléguer un disque ou une carte graphique à une machine virtuelle. Un autre axe d'amélioration consisterait à renforcer la légèreté du système, en se basant sur une configuration du noyau minimale, dans laquelle seuls certains pilotes seraient disponibles.

Ces perspectives illustrent qu'un nombre restreint de modifications conceptuelles est peut transformer fondamentalement la portée du projet, l'implémentation sous-jacente de ces modifications restant néanmoins nettement plus complexe que leur description conceptuelle.

Conclusion

Ce travail répond à une problématique identifiée à titre personnel : l'absence, parmi les solutions existantes, d'un système d'exploitation alliant déclarativité continue, intégration native de la conteneurisation et légèreté suffisante pour des déploiements modestes. Le travail de semestre ayant précédé ce travail de diplôme avait établi qu'aucune des solutions examinées, telles que NixOS ou Talos Linux, ne répondait simultanément à ces trois exigences. Le système développé au cours du présent travail comble ce vide, en s'appuyant sur les conclusions architecturales issues de cette analyse préalable.

La solution proposée est administrée selon un modèle d'administration entièrement déclaratif, reposant sur un unique fichier de configuration. Le système n'expose aucun accès interactif de type SSH ou console d'administration locale : l'API constitue l'unique surface d'administration, tant pour la soumission initiale de la configuration que pour toute modification ultérieure, ce qui garantit une prise en main simple et une intégration naturelle dans un contexte GitOps. Cette déclarativité repose sur un modèle de ressource homogène, dont la réconciliation est prise en charge par un orchestrateur centralisé, ce choix architectural privilégiant la coordination des dépendances entre ressources, au détriment d'une flexibilité de planification jugée secondaire au regard du périmètre du système. L'implémentation de cette architecture, détaillée au [chapitre 3, Implémentation](#), repose sur Rust pour l'ensemble des composants et sur Nix pour la chaîne de build, ce dernier choix garantissant la reproductibilité des artefacts produits.

Les résultats présentés au [chapitre 6, Résultats et Discussion](#) établissent que le système dépasse le stade d'une simple preuve de concept. Le déploiement d'une application complexe réelle, ainsi que la validation de la persistance des données à travers un redémarrage, démontrent une robustesse fonctionnelle suffisante pour des cas d'usage réels, et non uniquement pour des scénarios de démonstration simplifiés. Les mesures effectuées dans le [chapitre 4.3, Analyse de performance](#) confirment par ailleurs l'atteinte de l'objectif de légèreté fixé en introduction : le système démarre un conteneur en moins de 5.1 secondes lorsqu'une image doit être téléchargée, et ne requiert que 160 MiB de mémoire pour exécuter un serveur web minimal, un seuil sensiblement inférieur à celui des solutions comparables. Au regard de l'ensemble de ces éléments, les objectifs techniques fixés par l'énoncé du sujet sont considérés comme atteints.

Ce travail a également permis de mettre en application, dans un contexte concret, les notions de conteneurisation abordées durant la formation, contribuant à une compréhension plus approfondie de leur fonctionnement interne. Il a en outre permis d'approfondir l'usage de Nix comme système

de build, dont l’apport en matière de reproductibilité s’est révélé concluant, en dépit des temps de compilation élevés discutés au [chapitre 6.3, *Difficultés rencontrées*](#).

Le périmètre volontairement restreint retenu ici, celui du déploiement modeste et sans clustering, constitue une base sur laquelle les extensions envisagées, qu’il s’agisse de l’observabilité, du stockage ou d’une intégration plus poussée avec la conteneurisation, pourront être construites sans remettre en cause les choix architecturaux fondamentaux faits dans ce travail. S’agissant d’un projet avant tout personnel, le développement du système est appelé à se poursuivre au-delà du cadre de ce travail de diplôme, avec pour objectifs immédiats d’implémenter les fonctionnalités relatives à l’observabilité et au stockage.

Annexes

Annexe A

Utilisation de l'intelligence artificielle

Le modèle utilisé pour l'ensemble des interactions documentées ci-dessous est Claude Sonnet 4.6 Thinking, via Perplexity.

A.1. Recherche d'options de configuration manquantes du noyau Linux

Le problème rencontré est que Podman ne parvient pas à démarrer correctement des conteneurs, la `defconfig` du noyau ne contenant pas l'ensemble des fonctionnalités requises. La majorité des options manquantes ont été identifiées manuellement au préalable, mais les messages d'erreur obtenus lors de la configuration de `netavark` ne se sont pas révélés suffisamment explicites pour identifier les options restantes. Le fragment de configuration du noyau (limité aux modifications apportées à la `defconfig`), le message d'erreur obtenu et du contexte (`defconfig`, système embarqué, etc.), ont été transmis avec une demande visant soit à identifier l'option manquante, soit à fournir des sources pertinentes. En pratique, l'IA ne parvient pas à identifier l'option manquante, mais oriente effectivement la recherche vers le wiki de Gentoo, à partir duquel une autre page permet d'identifier une partie des options manquantes; seule l'option `CONFIG_NFT_FIB_IPV6` demeure introuvable par ce biais, l'IA ne l'ayant ni identifiée ni sourcée. Une recherche complémentaire, menée après cet échec, aboutit finalement à un commentaire publié sur le dépôt GitHub de `podman-compose`, fournissant l'option manquante. L'IA n'apporte, dans ce cas, qu'un résultat mitigé.

A.2. Amélioration du pipeline de build Nix pour Rust

Le problème rencontré est que, lors de la construction du workspace Rust au moyen de Nix et de Crane, toute modification apportée à l'une des crates entraîne la reconstruction de l'ensemble des crates, plutôt que des seules crates qui en dépendent effectivement. La configuration du workspace Cargo ainsi que les fichiers Nix ont été fournis, avec la description du problème. L'IA ne parvient ni à identifier la cause, ni à proposer une solution fonctionnelle. La cause réelle du problème tient au fait que l'ensemble des dépendances, y compris locales, est déclaré dans le `Cargo.toml` du workspace, ce qui invalide systématiquement la préparation des artefacts effectuée par Crane; le retrait de ces dépendances du workspace, au profit d'une déclaration du chemin directement dans chaque crate dépendante, résout le problème, moyennant l'ajout d'une option `deps` à la fonction de build afin d'inclure ces crates dans les sources. L'IA n'apporte, dans ce cas, aucune aide et constitue une perte de temps estimée à 30 minutes.

A.3. Mise en place de tests d'intégration Rust isolés par namespace

Le problème rencontré tient à la nature des paquets concernés, qui interagissent directement avec le noyau Linux, ce qui rend l'écriture de tests unitaires ou d'intégration délicate: ces tests modifient le noyau « hôte » et risquent de provoquer des dysfonctionnements, par exemple par l'ajout ou la suppression d'interfaces réseau, ou la modification de `/etc/resolv.conf`. Aucun travail préalable n'a été mené sur ce point. Les besoins ont été décrits et il a été demandé d'identifier les crates pertinentes pour ces usages. L'IA identifie effectivement une crate pertinente (`netns` [64]), qui, bien que ne correspondant pas exactement au besoin, permet de s'inspirer de son code source pour produire une implémentation propre au projet. L'IA apporte, dans ce cas, une aide effective, avec un gain de temps estimé à 15 minutes.

Annexe B

Exemple de configuration complète de l'OS

```
1  ---
2  schema: api
3  auth: "none"
4
5  ---
6  schema: network:dns
7  nameservers:
8    - 9.9.9.9
9
10 ---
11 schema: network:link
12 name: eth0
13 admin_up: true
14
15 ---
16 schema: network:dhcp
17 name: eth0
18
19 ---
20 schema: container:runtime
21 name: rootless
22 engine: podman
23 uid: 1000
24 gid: 1000
25 depends_on:
26   - network:dns
27   - network:route/eth0-dhcp
28
29 ---
30 schema: container:instance
31 name: http
32 image: docker.io/fredericarr/simple-http-server:latest
33 runtime: rootless
34 ports:
35   - container_port: 80
36     host_port: 8080
```

YAML YAML

Annexe C

Le langage Nix

Cette annexe présente les concepts fondamentaux de Nix utilisés dans le système de build présenté au [chapitre 3.8, *Système de build*](#). Le contenu est largement inspiré de <https://linuxvox.com/blog/an-overview-of-nix-os-architecture/>

C.1. Dérivations

Dans Nix, une dérivation représente une fonction pure (sans effet de bord et ne se basant que sur ses entrées), accompagnée des instructions nécessaires pour build un logiciel depuis ses sources afin de créer un artefact entièrement autonome, ou « self-contained » (l'ensemble des dépendances nécessaires à l'utilisation de l'artefact est inclus avec celui-ci, à la manière d'un conteneur). Les dérivations sont « évaluées », c'est-à-dire exécutées, dans un environnement complètement isolé. En outre, les dérivations étant des fonctions pures, produisant, pour des entrées identiques, des sorties identiques au bit près, leur résultat est mis en cache, de sorte que deux évaluations d'une même dérivation ne soient en réalité exécutées qu'une seule fois.

C.2. nixpkgs

Nixpkgs est le dépôt officiel de Nix, dans lequel les paquets sont mis à disposition sous forme de dérivations déjà construites et mises en cache; il s'agit de l'équivalent d'un dépôt APT, par exemple.

C.3. Le Nix Store

Le Nix Store est le répertoire unique dans lequel l'ensemble des résultats des dérivations Nix sont placés: `/nix/store`. Chaque dérivation y occupe un sous-répertoire dont le nom intègre un hash cryptographique dérivé des entrées de la dérivation correspondante, ce qui garantit qu'aucune dérivation ne partage de chemin avec une autre. Cette unicité des chemins permet à plusieurs versions d'un même logiciel de coexister sans jamais s'écraser mutuellement, et les dérivations y sont ajoutées ou retirées en tant qu'unités complètes, ce qui exclut toute installation partielle.

C.4. Nix comme système de build

En tant que système de build, Nix ne se limite pas au build d'artefacts isolés: il permet également de définir des environnements de développement reproductibles, activables au moyen de la commande `nix develop`, ainsi que d'exécuter une commande arbitraire au sein d'un tel environnement sans l'ouvrir de manière interactive, au moyen de `nix develop -c <commande>`. Cette dernière capacité

permet à un pipeline CI/CD de bénéficier exactement du même environnement que celui utilisé en développement local.

Bibliographie

- [1] Intel, « What Is a Trusted Platform Module (TPM)? ». Consulté le: 16 novembre 2025. [En ligne]. Disponible sur: <https://www.intel.com/content/www/us/en/learn/what-is-a-trusted-platform-module.html>
- [2] Docker Inc., « Docker Compose », Docker Documentation. Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://docs.docker.com/compose/>
- [3] The Linux Foundation, « Kubernetes ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://kubernetes.io/>
- [4] Hashicorp, « Terraform ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://www.hashicorp.com/en/products/terraform>
- [5] J. Holdsworth et A. Badman, « What is infrastructure as code (IaC)? », The 2026 Guide to DevOps. Consulté le: 14 mars 2026. [En ligne]. Disponible sur: <https://www.ibm.com/think/topics/infrastructure-as-code>
- [6] I. Red Hat, « Le GitOps, qu'est-ce que c'est ? ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://www.redhat.com/fr/topics/devops/what-is-gitops>
- [7] The NixOS Contributors, « Nix & NixOS ». Consulté le: 11 décembre 2025. [En ligne]. Disponible sur: <https://nixos.org/>
- [8] Sidero Labs, « Talos Linux ». Consulté le: 7 décembre 2025. [En ligne]. Disponible sur: <https://www.talos.dev/>
- [9] The Linux Kernel Organization, « The Linux Kernel Archives ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://www.kernel.org/>
- [10] Daniel Stenberg, « curl - command line tool and library for transferring data with URLs ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://curl.se/docs/manpage.html>
- [11] The Alpine Linux Development Team, « Alpine Linux - a security-oriented, lightweight Linux distribution ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://www.alpinelinux.org/>
- [12] Ralph Droms, « RFC 2131 - Dynamic Host Configuration Protocol ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://datatracker.ietf.org/doc/html/rfc2131>

- [13] The kernel development community , Éd., « Operational States », The Linux Kernel Documentation. Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://docs.kernel.org/networking/operstates.html>
- [14] Mintsuki and contributors., « Limine - Modern, secure, portable, multiprotocol bootloader and boot manager. ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://github.com/limine-bootloader/limine>
- [15] Red Hat, *Podman: A tool for managing OCI containers and pods*. (2026). Consulté le: 11 février 2026. [En ligne]. Disponible sur: <https://github.com/containers/podman>
- [16] The Rust Authors, « Rust by Example ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://doc.rust-lang.org/stable/rust-by-example/index.html>
- [17] The Rust Authors, « Std library types », Rust By Example. Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://doc.rust-lang.org/stable/rust-by-example/std.html>
- [18] The gRPC Authors, « gRPC - A high performance, open source universal RPC framework ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://grpc.io/>
- [19] The gRPC Authors, « Tonic - A Rust implementation of gRPC ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://github.com/grpc/grpc-rust>
- [20] The Tokio Authors, « Tokio ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://github.com/tokio-rs/tokio>
- [21] The Serde Authors, « Serde ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://github.com/serde-rs/serde>
- [22] Yongjiang Zhang, « Why were `required` and `optional` removed in Protocol Buffers 3? », StackOverflow. Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://stackoverflow.com/questions/31801257/why-were-required-and-optional-removed-in-protocol-buffers-3>
- [23] M. Kerrisk, « Capabilities », Linux Capabilities and Namespaces. Consulté le: 4 février 2026. [En ligne]. Disponible sur: https://man7.org/training/download/capns_caps_slides-mkerrisk-man7.org.pdf
- [24] The Rust Team, « Struct BTreeMap », The Rust Standard Library. Consulté le: 23 juillet 2026. [En ligne]. Disponible sur: <https://doc.rust-lang.org/1.97.1/std/collections/struct.BTreeMap.html>
- [25] The Rust Team, « Struct HashSet », The Rust Standard Library. Consulté le: 23 juillet 2026. [En ligne]. Disponible sur: <https://doc.rust-lang.org/1.97.1/std/collections/struct.HashSet.html>

- [26] The Rust Team, « Trait Drop », The Rust Standard Library. Consulté le: 23 juillet 2026. [En ligne]. Disponible sur: <https://doc.rust-lang.org/1.97.1/std/ops/trait.Drop.html>
- [27] The Rust Team, « Struct Mutex », The Rust Standard Library. Consulté le: 23 juillet 2026. [En ligne]. Disponible sur: <https://doc.rust-lang.org/1.97.1/std/sync/struct.Mutex.html>
- [28] The Linux man-pages project, « open(2) - Linux System Call ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://www.man7.org/linux/man-pages/man2/open.2.html>
- [29] The Linux man-pages project, « link, linkat - make a new name for a file ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://www.man7.org/linux/man-pages/man2/linkat.2.html>
- [30] The Linux man-pages project, « openat2 - open and possibly create a file ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://www.man7.org/linux/man-pages/man2/openat2.2.html>
- [31] The Wikipedia Authors, « Time-of-check to time-of-use ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: https://en.wikipedia.org/wiki/Time-of-check_to_time-of-use
- [32] The Linux man-pages project, « rnetlink - Linux routing socket ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://www.man7.org/linux/man-pages/man7/rnetlink.7.html>
- [33] The Rust Netlink Authors, « rnetlink - Rust crate for rnetlink protocol ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://docs.rs/rnetlink/latest/rnetlink/>
- [34] The smoltcp Authors, « smoltcp - A smol tcp/ip stack in Rust ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://github.com/smoltcp-rs/smoltcp>
- [35] fussybeaver, « Bollard - Docker daemon API in Rust ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://github.com/fussybeaver/bollard>
- [36] The kernel development community, « Squashfs 4.0 Filesystem ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://www.kernel.org/doc/html/latest/filesystems/squashfs.html>
- [37] Christoph Rohland, « Tmpfs - A file system which keeps all of its files in virtual memory ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://www.kernel.org/doc/html/latest/filesystems/tmpfs.html>
- [38] The kernel development community, « Overlay Filesystem ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://www.kernel.org/doc/html/latest/filesystems/overlayfs.html>
- [39] The Linux Foundation, « Filesystem Hierarchy Standard ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html

- [40] I. Free Software Foundation, « Cryptsetup is an open-source utility used to conveniently set up disk encryption ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://gitlab.com/cryptsetup/cryptsetup/blob/master/README.md>
- [41] The Clevis Contributors, « Clevis - An Automated Encryption Framework ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://github.com/latchset/clevis/>
- [42] The Rust for Linux Authors, « Rust Unstable features ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://rust-for-linux.com/unstable-features>
- [43] The QEMU Project Developers, « QEMU - A generic and open source machine emulator and virtualizer ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://www.qemu.org/>
- [44] The Linux man-pages project, « fork - create a child process ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://www.man7.org/linux/man-pages/man2/fork.2.html>
- [45] The Linux man-pages project, « namespaces - overview of Linux namespaces ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://www.man7.org/linux/man-pages/man7/namespaces.7.html>
- [46] The Nextest Contributors, « A next-generation test runner for Rust ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://nexte.st/>
- [47] The Linux man-pages project, « _exit, _Exit - terminate the calling process ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: https://www.man7.org/linux/man-pages/man2/_exit.2.html
- [48] GitLab Inc., « Get started with GitLab CI/CD ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://docs.gitlab.com/ci/>
- [49] The Rust Team, « Documentation tests », The rustdoc book. Consulté le: 23 juillet 2026. [En ligne]. Disponible sur: <https://doc.rust-lang.org/1.97.1/rustdoc/write-documentation/documentation-tests.html>
- [50] The Typstyle authors, « Typstyle - A beautiful and reliable code formatter for Typst. ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://github.com/typstyle-rs/typstyle>
- [51] Buf Technologies, « Buf - Modern Protobuf and gRPC ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://buf.build/>
- [52] The Rust Authors, « rustfmt - A tool for formatting Rust code according to style guidelines ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://github.com/rust-lang/rustfmt>

- [53] The Rust Authors, « Clippy - A collection of lints to catch common mistakes and improve your Rust code ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://github.com/rust-lang/rust-clippy>.
- [54] Casey Rodarmor, « Just a command runner ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://just.systems/man/en/>
- [55] Free Software Foundation, « GNU make ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://www.gnu.org/software/make/manual/make.html>
- [56] Frédéric Arroyo, « OS pour le déploiement d'infrastructures et services conteneurisés ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: https://gitedu.hesge.ch/flg_bachelors/ps/2025/container_os/-/raw/bachelor/final/arroyo-frederic-sp-2026.pdf
- [57] The Nix community projects, « nixos-anywhere - Install NixOS everywhere via ssh ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://github.com/nix-community/nixos-anywhere>
- [58] Robin Lambertz, « The `impl From<std::Instant>` for `smoltcp::Instant` seems buggy », smoltcp. Consulté le: 1 août 2026. [En ligne]. Disponible sur: <https://github.com/smoltcp-rs/smoltcp/issues/347>
- [59] Frédéric Arroyo, « fix: incorrect conversion from `std::time::Instant` to `Instant` », smoltcp. Consulté le: 1 août 2026. [En ligne]. Disponible sur: <https://github.com/smoltcp-rs/smoltcp/pull/1177>
- [60] Jiri Daněš, « Alignment with moby swagger definition for ContainerState.Status », Podman. Consulté le: 1 août 2026. [En ligne]. Disponible sur: <https://github.com/podman-container-tools/podman/issues/17728>
- [61] crawfordxx, « compat: map internal states to Docker equivalents in LibpodToContainer », Podman. Consulté le: 1 août 2026. [En ligne]. Disponible sur: <https://github.com/podman-container-tools/podman/pull/28390>
- [62] The Linux Foundation, « v6.0.0 », Podman. Consulté le: 1 août 2026. [En ligne]. Disponible sur: <https://github.com/podman-container-tools/podman/releases/tag/v6.0.0>
- [63] Sascha Grunert, « Podman 6.0: BoltDB removal and SQLite migration plan », nixpkgs. Consulté le: 1 août 2026. [En ligne]. Disponible sur: <https://github.com/NixOS/nixpkgs/issues/489717>
- [64] Andrew Cann, « netsim - a Rust library which allows you to ». Consulté le: 2 août 2026. [En ligne]. Disponible sur: <https://github.com/canndrew/netsim>